

ניווט מוצל להולכי רגל



סמינר בנות בת שבע יג' יד'



לוגו מכללה:

שם מכללה: בנות בת שבע

סמל מוסד: 741710

שם המכללה: בנות בת שבע

שם הסטודנטית: תמר שימל

ת.ז.: 215949371

שם המנחה: הגב' רחל טוג

תאריך הגשה: תמוז תשפ"ה

תוכן עניינים

5.....	הצעת פרויקט
15.....	מבוא
15.....	הרקע לפרויקט
15.....	תהליך המחקר
15.....	סקירת ספרות
16.....	אתגרים מרכזיים
16.....	הבעיה איתה התמודדתי:
16.....	הסיבות לבחירת הנושא:
16.....	מוטיבציה לעבודה:
16.....	על איזה צורך הפרויקט עונה?
17.....	הצגת פתרונות לבעיה:
17.....	מטרות ויעדים
18.....	אתגרים
18.....	מדדי הצלחה למערכת
18.....	רקע תיאורטי
18.....	מושגים עיקריים בתחום הדעת של הבעיה
20.....	נושאים רלוונטיים בתחום הדעת של הבעיה
21.....	סקירה מקצועית של הבעיה בעולם
22.....	נושאים רלוונטיים במדעי המחשב בהקשר לבעיה
23.....	תיאור פתרונות קיימים לבעיה
23.....	ניתוח חלופות מערכתי
25.....	תיאור החלופה הנבחרת + נימוקים
25.....	אפיון המערכת שהוגדרה/מוצעת
25.....	ניתוח דרישות המערכת
26.....	מודול המערכת
26.....	אפיון פונקציונלי
26.....	ביצועים עיקריים
26.....	אילוצים
27.....	תיאור הארכיטקטורה
27.....	הארכיטקטורה של הפתרון המוצע בפורמט של Top-Down level Design
28.....	תיאור הרכיבים בפתרון
28.....	תיאור תהליכים של מערכת ההפעלה
28.....	ארכיטקטורת רשת
29.....	תיאור API בארכיטקטורה
29.....	תיאור פרוטוקולי התקשורת
29.....	שרת- לקוח

30.....	תיאור תהליכי אבטחת מידע במערכת
30.....	תיאור התקיפה או ההגנה
31.....	תיאור הצפנות
31.....	למידת מכונה
31.....	תיאור התוכנה
31.....	פירוט API
32.....	סביבת עבודה
33.....	שפות תכנות
33.....	ניתוח ותרשים UML / Use cases של המערכת המוצעת
33.....	תיאור ה-UC העיקריים של המערכת
34.....	הצגת מקרה (Use Case) עבור כל הפונקציות העיקריות בפרויקט
35.....	מבני נתונים בהם השתמש (רשימה, מחסנית, תור וכו')
36.....	חישוב יעילות האלגוריתמים
36.....	הקשרים בין היחידות השונות
37.....	עץ מודולים
38.....	Use Case Diagram
39.....	רשימת Use Cases
39.....	UML
40.....	תרשים מחלקות
42.....	תיאור המחלקות המוצעות - מהכלל אל הפרט
46.....	תרשים מסכים המתאר את היררכיית המסכים והמעברים ביניהם (Screen flow diagram)
46.....	עבור כל מסך באפליקציה
46.....	מה תפקידו של כל מסך
46.....	תיאור של כל מסך
48.....	צילום של כל מסך
49.....	עבור כל אלמנט תצוגה - הסבר בתפקיד
51.....	הודעות למשתמש
51.....	ממשק משתמש
52.....	קוד התוכנית - על פי סטנדרטים בליווי תיעוד
52.....	קלט
53.....	פלט
54.....	פונקציות עיקריות
59.....	תיאור מסד הנתונים
59.....	תרשימי DSD
59.....	תרשימי ERD
61.....	מדריך למשתמש
62.....	בדיקות והערכה
62.....	מסקנות



63..... פיתוחים עתידיים

63..... ביבליוגרפיה

הצעת פרויקט

תיאור הפרויקט

האפליקציה מחשבת עבור המשתמש את המסלול המוצל ביותר ביחס לדרך שהמשתמש מוכן להאריך, בין נקודה A לנקודה B, כאשר נקודה A היא נקודת מוצא- תחילת המסלול המבוקש, ונקודה B היא יעדו של המסלול.

הפרויקט מיועד לכל אדם המעוניין לנווט במסלול הליכה עם חשיפה מופחתת לשמש. ישנם אנשים שמעוניינים ללכת בדרך עם חשיפה מינימלית לשמש משום שהיא מסנוורת, מחממת או פשוט לא נעימה עבורם, וישנם אנשים שהשמש לא רק מציקה להם, אלא היא מזיקה להם ממש לבריאותם כמו חולי לופוס- שיש להם רגישות לקרינה.

לסיכום המערכת נותנת מענה: לאנשים שמעדיפים להימנע מהליכה בשמש חזקה, ולאנשים שיש להם רגישות לקרינה.

על מנת לבנות מסלול מוצל, יש להבין כיצד צל נוצר וכיצד הוא משתנה לאורך זמן.

כיצד צל נוצר? הצל נוצר כתוצאה מחסימת קרני השמש על ידי עצם קבוע או עצם מתנייד. באפליקציה זו תהיה התייחסות לעצם קבוע: ומהו עצם קבוע? עצם שנמצא באופן יציב על המרחב ואלו הם: עצים, מבנים (בניין מגורים, בית פרטי, מרכזים מסחריים).

אילוץ נוסף לעצם קבוע: שיהיה מספיק גבוה (שיהיה יותר גבוה מאדם ממוצע). עצמים נמוכים אינם יוצרים צל משמעותי ולכן אינם נכללים בחישוב.

ולא תהיה התייחסות לעצם שהוא אינו קבוע - עצם מתנייד כמו רכב אדם וכדו'.

וכיצד הצל משתנה? מאפייני הצל אינם קבועים הם משתנים לאורך שעות היום, בין עונות השנה, לפי קו הרוחב של המקום ולפי גובה הפריטים המייצרים אותו.

בבוקר ובשעות אחר הצהריים השמש נמוכה, מה שיוצר צל ארוך פי כמה מאשר בשעות הצהריים. בקיץ השמש גבוהה יותר ויוצרת צל קצר, ואילו בחורף זווית השמש נמוכה יותר ולכן ניתן ליהנות מצל רב יותר. גם מיקומו הגאוגרפי של המשתמש משפיע: ככל שקרוב יותר לקו המשווה, זוויות השמש גבוהות יותר ככל שהוא צפוני או דרומי יותר, זוויות נמוכות יותר ממושכות לאורך היום ותהיה התייחסות לזווית השמש.

לסיכום, האפליקציה מתחשבת בשלושה גורמים מרכזיים: זווית השמש, עצמים קבועים, ומזג האוויר, ואינה מתחשבת בעצמים ניידים (כמו רכב, ואדם).

על פי נתונים אלו נוכל לחשב מסלול הליכה מוצל ואופטימלי.

תיאור הבעיה האלגוריתמית

בבעיה זו קיימת רשת רחובות גיאוגרפית (ה"מפה") עם קטעי דרך קבועים, ומודל צל דינמי המשתנה בהתאם לזמן, למיקום הגיאוגרפי, ולמזג האוויר. המטרה היא למצוא מסלול אופטימלי עבור הולך רגל יחיד (המשתמש) בין שתי נקודות נתונות (מוצא ויעד), תוך מזעור החשיפה לשמש ושהייה בצל לאורך המסלול.

המורכבות האלגוריתמית נובעת מהצורך לשלב מספר תחומים שונים לכדי חישוב אחד. ראשית, צריך לחשב את מיקום השמש בשמיים בזמן אמת, בעיה אסטרונומית ופיסיקלית קלאסית של מיקום חיזוי. בנוסף לכך את בעיית זיהוי אובייקטים שהיא בעיה קלאסית בראייה ממוחשבת למציאת מופעים ספציפיים של אובייקטים מתוך תמונה או וידאו. זה לא רק לקבוע אילו סוגי אובייקטים קיימים בסצנה, אלא גם היכן בדיוק הם ממוקמים.

בנוסף, קיימת בעיית ניתוב קלאסית, הדורשת בחירת מסלול מיטבי בין שתי נקודות, שהיא מציאת מסלול המאזן בין אורך הדרך לרמת הצל.

הקלט והפלט

• קלט:

- מיקום גיאוגרפי של נקודת המוצא (A) ונקודת היעד (B).
- תאריך, שעה נוכחית, ואזור זמן של המשתמש (לקביעת זווית השמש).
- נתוני מפה גיאוגרפיים.
- אילוף העדפה של המשתמש: משקל יחסי בין "מרחק הליכה" לבין "רמת הצל" (למשל, מוכנות להאריך את הדרך ב- $X\%$ עבור צל נוסף).
- נתוני מזג אוויר עדכניים המשפיעה על רמת הצל.

• פלט:

- סדרת נקודות ציון גיאוגרפיות המגדירות את המסלול המוצל והאופטימלי.
- זמן הליכה משוער וסך אורך המסלול.
- דוח חשיפה לשמש המציג את אחוז המסלול המוצל לעומת החשף לשמש (מדד אופטימיזציה)

דרישות

1. **חישוב הצל:** יש לחשב את מצב הצל (מוצל/חשוף) של כל קטע דרך ברשת באופן דינמי בהתאם לשעת היום העונה המיקום הגיאוגרפי של המשתמש.
2. **מגבלות גיאוגרפיות:** הניתוב חייב להתחשב רק בדרכים המוגדרות כנתיבי הליכה אפשריים (מדרכות, מעברי חצייה (תוך הימנעות ממכשולים פיזיים) בניינים, קירות).
3. **אילוף אופטימלי כפול:** האלגוריתם צריך למצוא את המסלול הטוב ביותר מבחינת שקלול של מרחק וצל.
4. **מגבלות זמן הליכה:** המסלול המוצע חייב לעמוד באילוף שהוגדר על ידי המשתמש.

רקע תיאורטי

נושא הניווט במרחב העירוני זוכה להתייחסות רחבה במחקר ובפיתוח מערכות טכנולוגיות, בעיקר בהקשר של תכנון מסלולים להולכי רגל. לצד שיקולים של מרחק וזמן, קיימת התעניינות בגורמים סביבתיים המשפיעים על חוויית ההליכה, כגון טמפרטורה, קרינה סולארית ונחות כללית. בעולם קיימות מספר גישות ופתרונות המתמודדים עם בעיות מקבילות או דומות, אם כי לא באופן משולב ומלא.

אחת הגישות הנפוצות היא מערכות ניווט מבוססות מפות עירוניות, כגון יישומי ניווט להולכי רגל, המציעים מסלולים קצרים או מהירים. מערכות אלו מתבססות על ייצוג גרפי של רשת הרחובות, כאשר כל קטע דרך משוקלל בעיקר לפי מרחק או זמן הליכה משוער. יתרון: זמינות גבוהה, חישוב מהיר ויכולת ניווט מדויקת במרחב העירוני. חיסרון: היעדר התחשבות בגורמים סביבתיים כגון חשיפה לשמש או אזורים מוצלים.

גישה שניה היא מחקרי נוחות תרמית עירונית (Urban Thermal Comfort) העוסקים בניתוח השפעת הסביבה הבנויה על תחושת החום והנוחות של הולכי רגל. מחקרים אלו משתמשים במודלים פיזיקליים של קרינה סולארית, טמפרטורה, רוח וצל, ומפיקים מדדים או מפות תרמיות של אזורים עירוניים. יתרון: ניתוח מעמיק של השפעת הצל והמבנה העירוני על תנאי ההליכה. חיסרון: פתרונות אלו הם לרוב מחקריים או תכנוניים, ואינם מיועדים לשימוש אישי בזמן אמת.

גישה שלישית מתמקדת במערכות סימולציה וניתוח צל עירוני, הנפוצות בעיקר בתחומי האדריכלות והתכנון העירוני. מערכות אלו מחשבות את הטלת הצל של מבנים בהתאם לזווית השמש, לשעת היום ולעונות השנה, ומשמשות להערכת איכות הסביבה העירונית. יתרון: יכולת חישוב מדויקת של אזורי צל על בסיס נתונים גאומטריים. חיסרון: היעדר שילוב עם מנגנוני ניתוב דינמיים והיעדר התאמה לניווט אישי.

מקורות מידע על עצמים היוצרים צל בפתרונות קיימים לניתוח סביבתי ולחישוב צל עירוני, המידע על עצמים היוצרים צל מתקבל ממספר מקורות עיקריים. מקור מרכזי הוא מאגרי מידע גאוגרפיים (GIS) הכוללים נתונים על מיקום מבנים, קווי מתאר ושימושי קרקע, ולעיתים גם נתוני גובה. מאגרים אלו נאספים ומנוהלים על ידי רשויות מקומיות וגופים ציבוריים, ומשמשים בסיס למודלים עירוניים שונים. בנוסף, נעשה שימוש במודלים תלת-ממדיים של ערים, המאפשרים ייצוג מדויק של גובה וצורת מבנים לצורך חישובי הצללה. גישה נוספת מבוססת על עיבוד תצלומי לוויין ונתונים חזותיים, תוך שימוש באלגוריתמים של ראייה ממוחשבת לזיהוי עצמים קבועים כגון מבנים ועצים. שילוב מקורות מידע אלו מאפשר מיפוי של עצמים בעלי פוטנציאל יצירת צל ומהווה תשתית לפתרונות בתחום ניתוח הסביבה העירונית.

לסיכום, הפתרונות הקיימים בעולם מתמודדים כל אחד עם היבט חלקי של הבעיה - ניווט, ניתוח סביבתי או סימולציה צל - אך אינם משלבים בין התחומים באופן אחיד. הפרויקט המוצע נועד להשלים פער זה באמצעות שילוב רעיוני של הגישות הקיימות, ובכך לתת מענה מקיף יותר לבעיית הניווט במרחב עירוני עם חשיפה לשמש.

בעיות דומות וגישות לפתרון

להלן מספר בעיות מתמטיות ואלגוריתמיות המעניקות השראה לפתרון הבעיה, לצד אלגוריתמים אפשריים:

בעיית חישוב זווית השמש (Angle Sun)

אחד האתגרים המרכזיים בתכנון מוצל הוא חישוב זווית השמש בזמן אמת, שכן מיקום השמש בשמיים משפיע ישירות על אורך הצל וכיוונו. אלגוריתמים מרכזיים בפתרון בעיה זו:

- משוואת זריחת השמש
- נוסחאות אסטרונומיות אנליטיות (SPA / Meeus)
- אפרימריסות נומריות (JPL / DE + Horizons)

בעיית זיהוי עצמים (Object Detection)

שיטות מתקדמות בתחום זה מאפשרות לזהות ולמפות עצמים סטטיים במרחב העירוני מתוך תמונות לוויין, מודלים או מסדי נתונים גיאוגרפיים. זיהוי אובייקטים מהווה שלב קריטי, שכן רק לאחר זיהוי מקומם, צורתם, וגובהם ניתן לחשב את הטלת הצל על פני השטח בהתאם לזווית השמש ולשעת היום ולמיקומו הגיאוגרפי.

אלגוריתמים מרכזיים בפתרון בעיה זו:

- YOLO
- Faster R-CNN
- SSD

בעיית חישוב מסלול (Path Planning)

בפרויקט הניווט המוצל, האילוץ המרכזי הוא מקסום הצל לאורך המסלול ביחס לדרך שהשתמש מוכן להאריך. בעיה זו דומה לבעיית Weighted Graph Search שבהן כל קטע דרך מקבל "משקל" או "עלות", ובמקרה הזה כמות הצל שיש בו. המשימה היא לייצר מסלול שהמשקל הכולל שלו גבוה ככל האפשר (יותר צל), תוך שמירה על מסלול תקין ומאוזן לדרך שהשתמש מוכן להאריך. אלגוריתמים מרכזיים בפתרון בעיה זו:

- A* (A-Star)
- Dijkstra's

גישות לפתרון

גישות לבעיית חישוב זווית השמש (Angle Sun):

1. משוואת זריחת השמש

- זהו אלגוריתם שמתבסס על משוואת זריחת השמש

$$\cos \omega_0 = -\tan \phi \times \tan \delta$$

כאשר:

ω_0 היא זווית השעה בזריחה כאשר הערך השלילי נלקח (או בשקיעה) כאשר הערך החיובי נלקח

ϕ היא קו הרוחב של הצופה על כדור הארץ.

δ היא זווית הנטייה השמשית.

- כדי לדעת איפה השמש נמצאת בדיוק בשמיים, נתבסס על משוואת זריחת השמש כך שנדע באיזה זווית נמצאת בדיוק השמש בשמים בשעה מסוימת, ביום, ובמיקום גיאוגרפי מסוים.
- **מעלות:** מאפשר חישוב מדויק של זווית השעה, פשוט, ללא קבצים חיצוניים, מתאים לריצה בזמן אמת.
- **חסרונות:** דיוק מוגבל לעומת מודלים אסטרונומיים מתקדמים (SPA / JPL), רגישות גבוהה לשגיאות בקלט.

2. נוסחאות אסטרונומיות אנליטיות (SPA / Meeus)

- שיטות מתמטיות מדויקות לחישוב מיקום גרמי - (Azimuth) בהתבסס אך ורק על נוסחאות אסטרונומיות סגורות, ללא טבלאות חיצוניות וללא תלויות ב-API.
- SPA - (Solar Position Algorithm)
- NREL - הוא אלגוריתם רשמי ומדויק של: (National Renewable Energy Laboratory)
- Meeus - (Jean Meeus) הוא אסטרונום שפיתח ספרי נוסחאות אסטרונומיות מדויקות (Astronomical Algorithms)
- המודלים שלו מהווים בסיס כמעט לכל חישוב אסטרונומי הנדסי בשנים האחרונות.
- בפרויקט הניווט המוצל יש צורך בחישוב זווית השמש שהוא אחד מגרמי השמיים.
- **מעלות:** דיוק גבוה מאוד (רמת שניות זוויתיות), SPA / Meeus עובדים לבד ללא צורך בקבצים חיצוניים או גישה לשרתים.
- **חסרונות:** רגיש לרמת הדיוק שגיאיה של אפילו דקה אחת יוצרת סטייה מורגשת בגובה השמש, מורכב יחסית למימוש ללא בקרה למרות שהאלגוריתם מהיר, הוא כולל עשרות נוסחאות.

3. אפיריסות נומריות (JPL / DE + Horizons)

- הן מאגרי נתונים אסטרונומיים מדויקים של מיקום ותנועת גרמי שמיים שנבנים על-ידי NASA/JPL באמצעות סימולציות פיזיקליות ארוכות טווח.
- בפרויקט הניווט המוצל יש צורך בחישוב זווית השמש שהוא אחד מגרמי השמיים ואנו יכולים להשתמש בממשקי API של שרתים אלו.
- **מעלות:** רמת הדיוק היא הגבוהה ביותר מבין כל השיטות להנדסה, מדויק יותר מכל מודל אנליטי כמו SPA או Meeus, מבוסס על פיזיקה ולא על קירובים מתמטיים בלבד.
- **חסרונות:** קושי בשינויים בזמן אמת, ממשק מורכב השימוש ב-Horizons וב-JPL דורשים הבנה טכנית טובה של הפורמטים שלא תמיד רשומים פשוט, המידע תלוי בזמנים וצריך להמיר נכון בין סוגי זמן שונים.

גישות לבעיית זיהוי עצמים (Object Detection):

1. YOLO - You Only Look Once

- משפחה של מודלים מהירים במיוחד שמבצעים זיהוי אובייקטים בתמונה במעבר יחיד על התמונה. נפוצים ביישומי זמן-אמת.
- בפרויקט הניווט המוצל יש צורך בזיהוי אובייקטים בזמן אמת לכל מסלול.
- **מעלות:** מהירות גבוהה מאוד, יכולת זיהוי גלובלית, פשטות יישומית וארכיטקטורה קומפקטית, ביצועים טובים במגוון רחב של גדלי אובייקטים, קהילה גדולה ותמיכה עשירה.
- **חסרונות:** קושי בזיהוי אובייקטים קטנים מאוד, הטיה לטובת צורות מסוימות של תיבות גבול, פחות מדויק ממודלים דו-שלביים - למרות המהירות, מודלים כמו Faster R-CNN לרוב מגיעים לדיוק גבוה יותר, במיוחד בזיהוי מורכב.

2. Faster R-CNN

- מודל דו-שלבי: בשלב ראשון מייצר הצעות לאזורים שבהם עשויים להיות אובייקטים, ובשלב שני מסווג אותם וממקם תיבות גבול (bounding boxes).
- בפרויקט הניווט המוצל יש צורך במיפוי מדויק לצורך חישוב כמות ושטח הצל בכל אזור (באמצעות bounding box או הערכת פרופורציות).
- **מעלות:** זיהוי תבניות מרחביות בצורה יעילה, פחות פרמטרים בהשוואה לרשתות fully-connected, עמידות לשינויים במיקום האובייקט, ביצועים מצוינים במשימות עיבוד תמונה, מנגנוני העמקה יעילים.
- **חסרונות:** דורש עוצמת חישוב גבוהה, דורש הרבה נתונים, פחות יעיל לזיהוי יחסים גלובליים, רגישות לרעש ונתונים לא מאוזנים, "קופסה שחורה" – קשה להסביר החלטות.

3. SSD – Single Shot MultiBox Detector

- מודל "שוט אחד" כמו YOLO, אך עם ארכיטקטורה שונה. מאוזן בין מהירות לבין דיוק, ומתאים במיוחד למכשירים ניידים ויישומים קלי-חומרה.
- בפרויקט הניווט המוצל יש צורך בזיהוי האובייקטים לצורך חישובי הצל במהירות לצד דיוק.
- **מעלות:** מהירות גבוהה- בדומה ל-YOLO, איזון טוב בין מהירות לדיוק- לא הכי מהיר ולא הכי מדויק אך שילוב יעיל במיוחד ליישומי תעשייה, רב-רזולוציה הוא מזהה אובייקטים גם בגדלים שונים, יחסית פשוט לאימון

וליישום, מתאים לחומרה חלשה.

- **חסרונות:** קושי בזיהוי אובייקטים קטנים, דיוק נמוך יחסית למודלים דו-שלביים, תלות חזקה בעוגנים (anchors), רגישות לשינויים בתנאי הסביבה, תלוי באיכות רשת הבסיס (backbone).

גישות לבעיית חישוב מסלול (Path Planning):

1. A* (A-Star)

- הוא אחד האלגוריתמים היעילים ביותר למציאת המסלול הקצר ביותר. האלגוריתם מתבסס על גרף כשבכל גרף יש צמתים וקשתות ועלות הקשת היא המרחק. האלגוריתם נמצא בשימוש נרחב מאוד בתחומים כמו ניווט (GPS), משחקי וידאו, רובוטיקה ותכנון תנועה.
 - האלגוריתם יחשב לפרויקט את הדרך המוצל ביותר ביחס לכמות הזמן שהמשתמש מוכן להאריך, ובמקום שעלות הקשת תהיה רק המרחק, היא תהיה גם "עלות חשיפה לשמש".
 - **יתרונות:** אופטימליות - כל עוד ההיוריסטיקה היא עקבית, הוא הרבה יותר מהיר Dijkstra's או מחיפוש ממצה, ניתן להתאים אותו בקלות לבעיות מורכבות, בדיוק כמו שצריך לפרויקט שצריך להגדיר את ה"עלות" כפונקציה של מרחק וגם צל.
 - **חסרונות:** יש לו צורך בנתונים מדויקים ומכוח החישוב הנדרש כדי להבטיח אופטימליות ועילות, החיסרון המרכזי של A* הוא התלות שלו באיכות ההיוריסטיקה.
- (היוריסטיקה- היא כלל אצבע, קיצור דרך מחשבתי או שיטה מעשית לפתרון בעיה, שאינה מבטיחה למצוא את הפתרון האופטימלי או הנכון ביותר, אך היא לרוב מהירה מספיק, קלה לשימוש וטובה מספיק ברוב המקרים).

2. Dijkstra's Algorithm

- הוא אחד מהאלגוריתמים המפורסמים ביותר למציאת המסלול הקצר ביותר ממקור יחיד בתוך רשת (גרף), מטרתו של Dijkstra's היא למצוא את המסלול הזול ביותר (בעל סכום המשקולות הנמוך ביותר) מקודקוד התחלה לכל שאר הקודקודים בגרף.
- האלגוריתם יחשב לפרויקט את הדרך המוצל ביותר ביחס לכמות הזמן שהמשתמש מוכן להאריך, ובמקום שעלות הקשת תהיה רק המרחק, היא תהיה גם "עלות חשיפה לשמש". (בדיוק כמו שאלגוריתם A* יכול להתאים לפרויקט)
- **מעלות:** מציאת פתרון אופטימלי מובטחת, פשוטה יחסית להבנה וליישום, עובד היטב ללא תלות ביעד.
- **חסרונות:** יעילות נמוכה בחיפוש גדולים (איטי בערים גדולות) ביחס ל-A*, אינו עובד עם משקולות שליליות, צורך בעיבוד מחדש לכל שאילתה.

הליכים עיקריים בהנדסת תוכנה

הפרויקט יכול לספר שלבים מרכזיים לצורך הפיתוח והיישום של מערכת הניווט המוצל. כל שלב מלווה בתכנון ובדיקות על מנת להבטיח מערכת מתפקדת ואפקטיבית.

1. איסוף נתונים גיאוגרפיים וניתוח ראשוני

- **איסוף נתוני מיקום מבנים:** איסוף נתוני רשת רחובות, קווי מתאר של בניינים ונתוני גובה.
 - **ניתוח דפוסי צל:** ניתוח המבנה העירוני כדי לזהות אזורים עם פוטנציאל צל גבוה או נמוך לאורך היום.
 - **הכנת "גרף" הניתוב:** המרת הנתונים הגולמיים למבנה נתונים של גרף (צמתים וקשתות) הניתן לעיבוד אלגוריתמי.
- ### 2. אפיון ויצירת מודל צל דינמי

- **בניית מודל הצל:** פיתוח מודל חישוב צל תלת-ממדי, המשלב את גובה המבנים עם זווית השמש.
 - **פיתוח מודל אסטרונומי:** חישוב מדויק של מיקום השמש (אזימוט וגובה) בכל זמן ומקום.
 - **תכנון שקלול עלויות:** אפיון מודל אלגוריתמי שימיר את נתוני הצל הפיזיקליים ל"עלות חשיפה לשמש" עבור כל קטע דרך בגרף.
- ### 3. פיתוח האלגוריתם המרכזי (הניתוב)

- **פיתוח אלגוריתם ניתוב אופטימלי:** פיתוח אלגוריתם שמטרתו למצוא את המסלול בעל העלות הנמוכה ביותר (המינימום חשיפה לשמש) בין נקודה A ל-B.
- **פיתוח מנגנון העדפות משתמש:** הטמעת לוגיקה המאפשרת למשתמש לשלוט באיזון בין "מסלול קצר" ל"מסלול מוצל יותר" (באמצעות שינוי משקולות).
- **בניית ממשק מפה:** פיתוח מודל גרפי המציג את המסלול המחושב על גבי המפה, לצורך המחשה ובקרה.

4. בדיקות והערכה

- **בדיקות ראשוניות (משרד):** בדיקת מודל הצל מול נתונים ידועים (עבור שעות שונות ביום) כדי לוודא דיוק חישוב.
- **הערכת איכות בשטח (בדיקות בטא):** בדיקות סימולציה ובחינת המערכת מול תרחישים מציאותיים על ידי הליכה בפועל במסלולים המוצעים והשוואה למציאות.
- **בדיקת ביצועי האלגוריתם:** בדיקת מהירות וחיסכון במשאבים של האלגוריתם, במיוחד בערים גדולות עם צמתים רבים.

תיאור טכנולוגיית הנדסת התוכנה

הפרויקט מבוסס על שילוב של מספר טכנולוגיות עיקריות, שמיועדות להבטיח פעולה יעילה, עדכניות ויכולת התמודדות עם כמויות מידע גדולות. מבנה המערכת והטכנולוגיות שנבחרו מסייעים להשגת מטרות הפרויקט בצורה מיטבית.

1. שפת תכנות:

- **C# (NET)** - צד השרת של הפרויקט מבוסס על C# באמצעות פלטפורמת .NET. ומספק תשתית יציבה ומהירה לעיבוד נתונים מורכבים וניהול מסד הנתונים. באמצעות C# ניתן לחשב את מסלולי ההליכה המוצלים בזמן אמת, עיבוד נתוני מזג אוויר וזוויות שמש, וניהול אבטחת המידע. שימוש ב-C# מאפשר כתיבת לוגיקה עסקית מסודרת, תחזוקה קלה והטמעה של מנגנוני אבטחה מתקדמים, תוך שמירה על ביצועים גבוהים ויציבות לאורך זמן.

2. React + Leaflet/Mapbox:

- **React + Leaflet/Mapbox** - נבחרה כטכנולוגיית צד לקוח לבניית ממשק משתמש אינטראקטיבי ודינמי. השילוב מאפשר הצגת מפה חיה עם מסלול ההליכה, סימון אזורים מוצלים מול אזורים חשופים לשמש, ושינויים בזמן אמת בהתאם לזווית השמש וגובה מבנים. React מנהלת את הלוגיקה והאינטראקציות, בעוד Leaflet או Mapbox מספקות כלים למיפוי מתקדם, שכבות מידע ואנימציות להצגת הצל לאורך המסלול.

3. אינטגרציה ותקשורת עם מקורות מידע חיצוניים:

- האפליקציה תשלב נתונים שמתקבלים ממערכות חיצוניים כגון נתוני מזג אוויר ותצלומי לוויין. הנתונים מתקבלים באמצעות API מובנים ומוזנים אל צד השרת - C# לצורך חישוב מסלולים, סימון אזורים מוצלים והצגת עדכונים בזמן אמת למשתמש. מבנה התקשורת מאפשר עיבוד מהיר של מידע משתנה.

ניהול מסד נתונים

PostgreSQL – היא נבחרה לניהול מסד הנתונים אשר יכיל מידע על המבנים, מיקום השמש, המסלולים, וכן על המשתמשים. בעקבות בחירת PostgreSQL המערכת תהנה ממבנה רלציוני מסודר, תמיכה במידע גאוגרפי (PostGIS), ביצועים גבוהים, יכולת עבודה עם כמויות גדולות של נתונים בזמן אמת.

יתרונות השילוב הטכנולוגי

השילוב בין C# לתהליכי צד השרת לבין React + Leaflet/Mapbox בממשק המשתמש המספק פתרון חזק לניהול המערכת, תוך הבטחת חווית משתמש אינטראקטיבית, תצוגת מסלול מפורט ועדכונים בזמן אמת. מסד נתונים PostgreSQL עם תמיכה ב-PostGIS, מבטיח ניהול אמין של מידע גאוגרפי ושליפה מהירה לפי הצורך. בסיום הפיתוח, המערכת תספק למשתמש תכנון מסלולי הליכה מוצלים עם אפשרות לבחור מסלול מוצל לבין מסלול קצר יותר ופחות מוצל, עדכני ומדויק.

פרוטוקולי תקשורת

HTTPS הוא פרוטוקול תקשורת מאובטח להעברת מידע בין דפדפן לשרת. הוא מצפין את הנתונים באמצעות TLS, וכך מבטיח שהמידע שעובר נשאר מוגן מפני יירוט או שינוי.

הליכים עיקריים באבטחת מידע

1. הגנה על שלמות המידע במאגר הנתונים

- **מנגנוני ולידציה (Validation):** כל שינוי במידע הגיאוגרפי עובר בדיקות תקינות כדי לוודא שהנתונים נכונים ומלאים.
- **הרשאות גישה מדורגות:** מניעת שינויים לא מורשים על ידי שימוש בהרשאות שונות למשתמשים ובקורות גישה במערכת (PostGIS/PostgreSQL).
- **ניהול לוגים (Logging):** שמירה על רשומות פעילות ומעקב אחר שינויים במבקרים כדי לאתר ניסיונות שינוי או מחיקה.

2. מניעת זיוף נתונים המשפיעים על חישוב הצל

- **אימות נתונים חיצוניים:** חסימת אפשרות להזנת נתונים חיצוניים שלא עברו אימות.
- **אימות צד שרת (Server-side Validation):** בדיקה שכל נתוני השמש (Sun Angle) והנתונים הגיאוגרפיים נכונים ואמינים.
- **בדיקות עקביות (Consistency Checks):** בדיקה של קשרי תלות בין גובה, מיקום ומבנה כדי לוודא שאין סתירות.

3. שמירה על פרטיות נתוני משתמש

- **הצפנת נתונים:** הצפנת נתוני משתמשים, מסלולים והעדפות אישיות בבסיס הנתונים.
- **הרשאות מחמירות:** הגבלת גישה לנתונים אישיים בהתאם למדיניות הרשאות מחמירה.
- **עמידה בתקני פרטיות:** הבטחת עמידה בתקני פרטיות.

4. הגנה על תקשורת בין הלקוח לשרת

- **תקשורת מאובטחת:** שימוש בתקשורת מוגנת בין הלקוח לשרת כדי למנוע גישה לא מורשית או התערבות בנתונים.

לוח זמנים

- ספטמבר-נובמבר: ניתוח, תכנון, וכתובת הצעת הפרויקט.
- דצמבר: בחירת מודל חישוב השמש והצל, ותכנון ארכיטקטורת המערכת.
- ינואר-מרץ: פיתוח מודל חישוב זווית השמש, בניית מודל חישוב הצל לכל קטע דרך והכנת גרף הניתוב.
- אפריל: פיתוח אלגוריתם הניתוב האופטימלי, ופיתוח צד לקוח.
- מאי: הרצה, בדיקות סימולציה, בדיקות שטח ואינטגרציה מלאה של כלל רכיבי המערכת. יוני: הכנה סופית, והגשה.

מבוא

הרקע לפרויקט

ישראל נמצאת באחת האזורים החמים בעולם, ובעונות הקיץ טמפרטורות האוויר מגיעות ל- 35-40 מעלות צלזיוס.

עבור הולכי רגל, החשיפה לקרינת השמש הישירה יכולה להיות אי נוחות, אך היא יכולה גם לסכן בריאות. אנשים הסובלים ממחלת הלופוס, אלרגיה לשמש, לחץ דם גבוה, בעיות לב, ומצבים רפואיים נוספים, נאלצים להגביל את זמן חשיפתם לשמש. כמו כן, ילדים, קשישים ומטיילים עם עגלות מחפשים מסלולים מוצלים כדי להפוך את ההליכה לנסבלת ובטוחה.

מערכות ניווט קיימות (Google Maps, Waze, Apple Maps) מוצאות את המסלול הקצר ביותר או המהיר ביותר - אך אף אחת מהן אינה מתחשבת ברמת הצל לאורך המסלול. הפרויקט "ניווט מוצל להולכי רגל" (ShadyWay) נועד למלא פער זה: מערכת שמחשבת את המסלול המוצל תוך התחשבות בכמות הזמן שהמשתמש מוכן להעריך בין שתי נקודות, תוך התחשבות בזמן הנוכחי, גובה מבנים וכיסוי עצים.

תהליך המחקר

לפני שנגעתי בשורת קוד אחת, יצאתי לשטח.

הבנתי שכדי לבנות מערכת שמחשבת צל - אני חייבת קודם כל להבין את הצל. יצאתי לאחד הבניינים ליד ביתי בשלוש שעות שונות: בוקר, צהריים וערב, ובדקתי כמה צל הוא מטיל? לאן הצל מופנה? ואיך הוא משתנה לאורך היום? ראיתי בעצמי את מה שהפיזיקה מלמדת: בבוקר הצל ארוך ומופנה מערבה, בצהריים קצר כמעט ישיר, ובערב מתארך שוב אך לכיוון מזרח.

לאחר מכן בחנתי - על אילו גורמים סביבתיים ניתן לסמוך כמקורות צל קבועים? הגעתי למסקנה שרק עצמים קבועים ובעלי גובה מספיק - מבנים ועצים - יוצרים צל אמיתי ומשמעותי. רכבים, אנשים ואלמנטים ניידים אחרים אינם ניתנים לחיזוי ולכן אינם רלוונטיים בפרויקט הנוכחי.

לאחר שהעמקתי בצל חשבתי על פרקטיקת הדבר, שאלתי את עצמי שאלות ובדקתי אותם:

השאלה הראשונה - כיצד אמצא את המסלול הטוב ביותר עבור המשתמש? כאן נכנס עולם אלגוריתמי הניתוב - חקרתי כיצד מייצגים רשת רחובות כגרף, וכיצד מוצאים בו את המסלול האופטימלי תוך התחשבות גם במרחק וגם ברמת הצל.

השאלה השנייה - כיצד אדע בדיוק היכן השמש נמצאת בשמיים בכל רגע נתון? גיליתי שתחום האסטרונומיה כבר פתר בעיה זו לפני עשרות שנים: קיימים אלגוריתמים מתמטיים מדויקים שמחשבים את מיקום השמש לפי תאריך, שעה ומיקום גיאוגרפי.

השאלה השלישית - מאיפה אשיג את נתוני מקורות הצל? חקרתי מאגרי מידע גיאוגרפיים פתוחים ונתוני לוויין כדי למצוא כיסוי מספיק של מבנים ועצים בישראל.

לאחר שאספתי את כל הנתונים הללו והבנתי את הפתרונות לכל שאלה, עמדתי בפני האתגר של חיבורם לכדי מערכת אחת. למדתי על Onion Architecture גישת ארכיטקטורת תוכנה המאורגנת בשכבות ריכוזיות, כאשר הלוגיקה העסקית נמצאת במרכז ואינה תלויה בתשתית החיצונית. גישה זו הייתה מתאימה לפרויקט שלי, שבו שכבות שונות (חישוב שמש, גרף ניתוב, מסד נתונים) צריכות לפעול יחד מבלי להיות תלויות אחת בשנייה.

סקירת ספרות

- אסטרונומיה וחישוב שמש:

ערך "מערכת השמש" מתוך המכלול (האנציקלופדיה העברית החופשית) - סיפק רקע

אסטרונמי בעברית שעזר להבין את מסלול השמש, זוויות הזריחה והשקיעה, ומושגי בסיס כמו אזימוט וגובה. הערך המוסף: הבנה שפתית ומושגית שאפשרה לי לקרוא ולהבין מקורות טכניים בנושא חישוב מיקום השמש ביתר קלות.

- נתונים גיאוגרפיים:
מפת OpenStreetMap של ירושלים (openstreetmap.org) - שימשה לאימות חזותי של נתוני הגרף שנשלפו דרך Overpass API. הצגה ישירה על המפה אפשרה לי לוודא שהדרכים, הצמתים והמבנים נשלפים כנדרש. הערך המוסף: כלי ויזואלי לבדיקת נכונות הנתונים הגיאוגרפיים. תיעוד OpenStreetMap - תיעוד הפורמט הגיאוגרפי של OSM. עזר לי להבין אילו תגיות קיימות עבור מבנים, רחובות וגובה, וכיצד לנתח את ה-XML בצורה נכונה.
- תיעוד Google Earth Engine (developers.google.com/earth-engine) - התיעוד הרשמי של ממשק ה-API לנתוני לוויין. עזר לי להבין כיצד לשלוף נתוני גובה צמחייה לפי קואורדינטות, ואת מבנה התשובה המוחזרת. הערך המוסף: הבנת הממשק ללא ניסוי וטעייה - חיסכון משמעותי בזמן.
- אלגוריתמים:
ערך "אלגוריתם דייקסטרה" מתוך המכלול - הסבר בעברית של האלגוריתם עם דוגמה ויזואלית. עזר לי להבין את שיטת ה"הרפיה" (Relaxation) בצורה אינטואיטיבית לפני שצללתי לקוד. הערך המוסף: ודאות שהבנתי את המנגנון הבסיסי נכון לפני מימוש.
- סרטון הסבר על אלגוריתם דייקסטרה (YouTube) - הדגמה ויזואלית של האלגוריתם על גרף בדוגמה שלב אחר שלב. עזר לי לעקוב אחרי ה-Priority Queue ומנגנון בחירת הצומת הבא. הערך המוסף: הבנה ויזואלית שלא ניתן לקבל מטקסט בלבד.

אתגרים מרכזיים

הבעיה איתה התמודדתי:

הבעיה המרכזית הייתה שילוב של שלושה עולמות שונים לכדי מנגנון אחד:

1. גיאומטריה גיאוגרפית - חישוב מרחקים ומיקומים על פני כדור הארץ.
2. אסטרונומיה - חישוב מדויק של מיקום השמש בכל שעה ומקום.
3. אלגוריתמיקה - מציאת מסלול אופטימלי בגרף גדול.

הסיבות לבחירת הנושא:

הפרויקט נולד מתוך חוויה אישית ביתית.

אימי עברה טיפול פנים קוסמטי יקר, ולאחריו נאסר עליה לצאת לשמש - כל חשיפה לקרינה ישירה הייתה עלולה לגרום נזק לעורה. בתקופה ההיא, כל יציאה מהבית הפכה לאתגר: היא לא יכלה לדעת מראש אילו קטעי הרחוב שטופי שמש ואילו מוצלים, ולכן פשוט נמנעה מלצאת לרחוב כלל. ראיתי כיצד הגבלה זו פגעה בשגרת חייה - ביצוע קניות, ביקורים אצל שכנים - כל אלו הפכו לבלתי אפשריים.

בחרתי בפרויקט זה כי הוא פותר בעיה אמיתית שקשורה לאנשים רבים, במיוחד לאנשים עם מצבים רפואיים המגבילים חשיפה לשמש. הפרויקט משלב ידע ממדעי המחשב, פיזיקה ומתמטיקה.

מוטיבציה לעבודה:

בני משפחה הנמנעים מהיחשפות לקרינת שמש, ראיתי מקרוב כמה קשה לתכנן מסלולי הליכה עם חשיפה מינימלית לשמש. רציתי ליצור כלי שיסייע להם ולאחרים.

על איזה צורך הפרויקט עונה?

הפרויקט נותן מענה לצורך בניווט מותאם סביבה - לא רק מרחק אלא גם רמת הנוחות הסביבתית.

הצגת פתרונות לבעיה:

במהלך המחקר בחנתי מספר גישות לכל אחת מהבעיות המרכזיות, כולל גישות שנפסלו בהמשך:

א. חישוב מיקום השמש:

- אלגוריתם SPA של NREL - נבדק. דיוק גבוה מאד (עד שניות זוויתיות), אך מימוש מורכב הכולל עשרות נוסחאות ביניים.
- אלגוריתם ("Astronomical Algorithms") - Meeus - נבחר. דיוק מספיק לצרכי הפרויקט, מימוש פשוט יחסית, עובד ללא תלויות חיצוניות.

ב. מקורות נתוני מבנים ועצים:

- Google Open Buildings v3 - נשלל. אינו מכסה את ישראל, ואין בו נתוני גובה מבנה.
- ניתוח תמונות לוויין עם YOLO - נשלל. מורכבות מימוש גבוהה מאד, דורש משאבי חישוב כבדים, ואין ערך מוסף ביחס לפתרונות הפשוטים יותר.
- OSM (OpenStreetMap) - נבחר למבנים. נתונים חנימיים, כיסוי מלא של ישראל, כולל גובה מבנים בחלק מהמקרים.
- ETH Canopy Height דרך Google Earth Engine - נבחר לעצים. נתוני גובה צמחייה ברזולוציה של 10 מטר, גלובלי, ללא עלות.

ג. אלגוריתם ניתוב:

- A* - נבדק. יעיל מאד לגרפים גדולים אך דורש פונקציית היוריסטיקה מותאמת שאינה טריוויאלית כאשר המשקל כולל גם צל.
- Dijkstra - נבחר. מבטיח פתרון אופטימלי בגרפים ממושקלים, מימוש פשוט, ומתאים לגודל הגרפים בפרויקט.

מטרות ויעדים

מטרות הפרויקט:

המטרה העיקרית של ShadyWay היא לספק לכל משתמש מסלול הליכה אופטימלי המשלב שני קריטריונים: מרחק וצל, כאשר המשתמש מגדיר את העדפתו ביניהם.

המטרות המשניות:

- מתן מסלול שמשלב מבנים ועצים כמקורות צל.
- התחשבות בזמן הנוכחי (שינויי צל לאורך היום).
- שמירת היסטוריית מסלולים עבור כל משתמש.
- ממשק משתמש נוח עם הצגת המסלול על מפה.

יעדים טכניים:

- חישוב מיקום השמש בדיוק של פחות מ- 0.01° לכל מיקום ושעה.
- בניית גרף מ-OSM ועיבוד קטע דרך של 1 ק"מ בפחות מ-5 שניות.
- תמיכה בהעדפת צל גמישה (1.0-2.0).
- RESTful API עם אימות JWT.
- מוד נתונים PostgreSQL עם PostGIS לשמירת מסלולים.

אתגרים

אתגרים טכניים:

1. דיוק חישוב מיקום השמש:
אלגוריתמים פשוטים לחישוב מיקום שמש מוסיפים שגיאה של 2° - 5° שמוביל לחישוב צל שגוי.
פתרון: יישמתי אלגוריתם Meeus מלא הכולל 14 שלבי חישוב ותיקון שבירה אטמוספירית.
2. נתוני גובה מבנים:
OSM, רוב המבנים אינם מסומנים עם גובה מדויק.
פתרון: היררכיית חילוץ גובה: ברירת מחדל 10 מ' $\rightarrow$ building: levels $\times$ 3 $\rightarrow$ height מ'.
3. נתוני עצים:
OSM בישראל, מרבית העצים אינם ממופים. OSM אינו מכסה עצי יצר ורחבות עירוניות.
פתרון: שימוש ב- ETH Global Canopy Height דרך GEE REST API - נתוני לוויין ב- 10 מ' רזולוציה.
4. ביצועי הגרף:
בניין גרף קטע של 1 ק"מ כולל מאות צמתים ואלפי חישובי צל.
פתרון: sampling של 5 נקודות בלבד לכל קטע, מרדיוס חיפוש מוגבל ל-60 מטר.
5. אינטגרציה בין שכבות:
תיאום בין שכבת ה-API, השירותים, וה-DB תחת Onion Architecture מחייב הגדרה מדויקת של ממשקים (interfaces).

אתגרים באיסוף הנתונים:

- XML של OSM יכול להיות גדול (עשרות MB לאזור עירוני) - דרשה חישוב streaming.
- גישה ל-GEE דורשת Service Account מוגדר עם הרשאות מתאימות.

מדדי הצלחה למערכת

1. דיוק חישוב השמש- שגיאה מקסימלית של פחות מ- 0.05° בהשוואה לתוכנת USNO Sky Calculator.
2. זמן תגובה- חישוב מסלול בתוך 10 שניות לאזור עירוני של 2×2 ק"מ.
3. יעילות ניצול צל- המסלול המוצל יענה לדרישת המשתמש.
4. אחוז כיסוי נתונים- גרף הניתוב יכלול את כל הדרכים ברזולוציה של 1 ק"מ.
5. ניהול משתמשים- הרשמה, התחברות ושליפת היסטוריה פועלות ללא שגיאות.

רקע תיאורטי

מושגים עיקריים בתחום הדעת של הבעיה

גרף (Graph):

גרף $G = (V, E)$ הוא מבנה מתמטי המורכב מקבוצת צמתים (Vertices) וקבוצת קשתות (Edges) המחברות ביניהם. בפרויקט זה:

- כל צומת = צומת דרך (רחוב/מדרכה) עם קואורדינטות GPS (lat, lon)
- כל קשת = קטע דרך בין שני צמתים עם שני מאפיינים:
 1. מרחק בפועל (מטרים).
 2. אחוז הצל על הקטע (0-100%).

מדובר בגרף מכון וממושקל (Directed Weighted Graph), כאשר ייתכן שניתן לעבור בכביש בכיוון אחד בלבד (רחוב חד-סטרי).

עלות קשת אפקטיבית:

הנוסחה המשמשת לחישוב עלות הקטע היא:

$$\text{עלות} = \text{מרחק} \times (1 + (1 - \text{צל}/100) \times (\text{העדפה} - 1))$$

דוגמה: קטע של 100 מ' עם 50% צל, והמשתמש בחר העדפה=1.5:

$$\text{עלות} = 100 = (0.5 \times (0.5 - 1) + 1) \times 1.25 \times 100 = 125$$

קטע של 100 מ' עם 100% צל:

$$\text{עלות} = 100 = (0.5 \times 0 + 1) \times 100$$

כלומר: קטע מוצל לחלוטין עולה בדיוק כמו המרחק האמיתי שלו, ואילו קטע שחשוף לשמש עולה יותר.

צל (Shadow):

קיצור קרינת השמש הישירה על ידי חסימה פיזית של גוף אטום. צל נוצר כאשר עצם קבוע- מבנה או עץ- ממוקם בין מקור האור (השמש) לנקודה הנמדדת. בפרויקט זה עצמים ניידים (רכבים, אנשים) אינם נכללים בחישוב מכיוון שמיקומם אינו ניתן לחיזוי.

אזימוט השמש (Solar Azimuth):

כיוון השמש ביחס לנקודת המדידה, נמדד בדרגות עם כיוון השעון:

0° = צפון, 90° = מזרח, 180° = דרום, 270° = מערב.

האזימוט קובע לאיזה כיוון "נוטה" הצל - הצל תמיד בכיוון הפוך לשמש (azimuth + 180°).

לדוגמה: שמש ממזרח (90°) → צל מוטל מערבה (270°).

גובה השמש (Solar Elevation):

הזווית שבין קרן השמש לבין מישור האופק, נמדדת ב-0° - 90°.

Elevation = 0°: השמש על האופק (שקיעה/זריחה) - צל ארוך מאד (כמעט אינסופי).

Elevation = 90°: השמש ישירות מעל הראש - אין צל כלל.

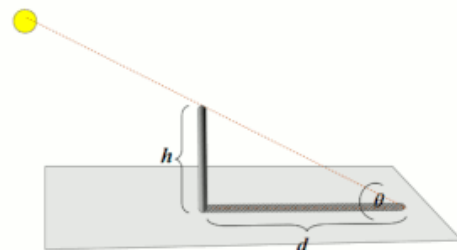
בפרויקט: ערך מינימלי של 5° מוגדר כדי למנוע חלוקה ב-0 בחישוב.

אורך צל (Shadow Length):

אורך הצל שמבנה מטיל מחושב לפי:

$$\text{shadow_length} = H / \tan(\text{elevation})$$

כאשר H = גובה האובייקט (מטרים), elevation = גובה השמש (רדיאנים).



נקודת GPS (GPS Coordinate):

ייצוג מיקום גיאוגרפי בשני ערכים:

- 1) Latitude (קו רוחב): מדרום לצפון, טווח -90° עד $+90^{\circ}$. ירושלים $\approx N31.77^{\circ}$
- 2) Longitude (קו אורך): ממערב למזרח, טווח -180° עד $+180^{\circ}$. ירושלים $\approx E35.21^{\circ}$

כל צומת בגרף, כל מבנה וכל עץ מיוצגים בפרויקט כזוג (lat, lon).

נושאים רלוונטיים בתחום הדעת של הבעיה

חישוב מיקום השמש - אלגוריתם Meeus

מיקום השמש משתנה לכל מיקום גיאוגרפי ושעת יום. חישוב מיקום השמש הוא בעיה אסטרונומית קלאסית. בפרויקט יישמנו את אלגוריתם Meeus שהוא הבסיס לרוב יישומי האסטרונומיה ההנדסיים.

שלבי האלגוריתם (14 שלבים):

שלב 1: תאריך יוליאני (Julian Day - JD)

$$JD = \text{floor}(365.25 \times (Y+4716)) + \text{floor}(30.6001 \times (M+1)) + D + B - 1524.5$$

שלב 2: מאה יוליאנית (Julian Century - JC)

$$JC = (JD - 2451545.0) / 36525.0$$

שלב 3: אורך ממוצע של השמש (L_0):

$$L_0 = 280.46646 + JC \times (36000.76983 + JC \times 0.0003032)$$

שלב 4: אנומליה ממוצעת (M):

$$M = 357.52911 + JC \times (35999.05029 - JC \times 0.0001537)$$

שלב 5: משוואת המרכז (C) - תיקון לאלפסה:

$$C = (1.914602 - JC \times (0.004817 + 0.000014 \times JC)) \times \sin(M) + (0.019993 - 0.000101 \times JC) \times \sin(2M) + 0.000289 \times \sin(3M)$$

שלב 6: אורך אמיתי: $\text{sun_lon} = L_0 + C$

שלב 7: תיקון אברציה:

$$\omega = 125.04 - 1934.136 \times JC$$

$$\lambda = \text{sun_lon} - 0.00569 - 0.00478 \times \sin(\omega)$$

שלב 8: נטיית מישור המילקה (ϵ):

$$\epsilon_0 = 23^{\circ} + (26 + (21.448 - JC \times (46.815 + JC \times (0.00059 - JC \times 0.001813))) / 60) / 60$$

$$\epsilon = \epsilon_0 + 0.00256 \times \cos(\omega)$$

שלב 9: עליה ישירה ונטייה:

$$RA = \text{atan2}(\cos(\epsilon) \times \sin(\lambda), \cos(\lambda))$$

$$\delta = \text{asin}(\sin(\epsilon) \times \sin(\lambda))$$

שלב 10: זמן כוכבי גריניץ' (GMST):

$$GMST = 280.46061837 + 360.98564736629 \times (JD - 2451545.0) + JC^2 \times (0.000387933 - JC / 38710000)$$

שלב 11: זווית שעה מקומית: $LHA = GMST - RA \times 15$ - קו אורך

שלב 12: גובה השמש:

$$\sin(\text{elevation}) = \sin(\phi) \times \sin(\delta) + \cos(\phi) \times \cos(\delta) \times \cos(LHA)$$

שלב 13: תיקון שבירת אטמוספירה (עבור $\text{elevation} > 5^{\circ}$):

$$\text{refraction} = (58.1 / \tan(\text{elev}) - 0.07 / \tan^3(\text{elev}) + 0.000086 / \tan^5(\text{elev})) / 3600$$

שלב 14: אזימוט:

$$az = \text{atan2}(\sin(LHA), \cos(LHA) \times \sin(\phi) - \tan(\delta) \times \cos(\phi)) + 180^\circ$$

תוצאה: Azimuth (0° – 360°), Elevation (-90° – $+90^\circ$), IsDaytime (elevation > 0°)

מודל הטלת הצל

כאשר השמש נמצאת בגובה elevation ובכיוון azimuth, מבנה בגובה H מטיל צל ב:

כיוון הצל: $azimuth + 180^\circ$

אורך הצל: $H / \tan(\text{elevation})$ - מוגבל ל-elevation מינימלי 5°

מודל חישוב בפרויקט - 5 נקודות דגימה לכל קטע דרך:

לכל נקודה בודקים מבנים ועצים בטווח 60 מטר:

- מבנה בחרוט הצל $100\% \rightarrow (dist \leq \text{shadow_length} \text{ AND } angle \leq 30^\circ)$ צל.
- עץ בחרוט + רדיוס צמרת $\rightarrow 60\%$ צל.
- ממוצע $\times$ מקדם עננות = אחוז צל סופי לקטע.

OpenStreetMap (OSM)

OSM הוא פרויקט מיפוי עולמי שיתופי ("ויקיפדיה של המפות"), חינמי ופתוח.

מבנה הנתונים: Node (נקודה), Way (קו/פוליגון), Relation (קבוצה).

תגיות רלוונטיות:

- $highway=footway/residential \leftarrow$ דרך להליכה
- $building=yes \leftarrow$ מבנה
- $height=15 \leftarrow$ גובה במטרים
- $building:levels=4 \leftarrow$ מספר קומות (3 מ' $\times$ כל קומה)

היררכיית חילוץ גובה: ברירת מחדל 10 מ' $\rightarrow 3 \times \text{building: levels} \rightarrow \text{height}$ מ'.

Google Earth Engine דרך ETH Global Canopy Height

ETH Global Canopy Height (2020) - נתון לוויין של אוניברסיטת ETH Zurich:

גובה הצמחייה בכל נקודה על פני כדור הארץ, רזולוציה 10 מטר.

שם הנכס: `ETH_GlobalCanopyHeight_2020_10m_v1/nlang/ users`.

גישה דרך Google Earth Engine (GEE) - פלטפורמת ענן לניתוח נתוני לוויין.

שיטה: `Image.sample` - ממיר רסטר לנקודות, כל נקודה עם ערך גובה.

גובה > 2 מ' $\leftarrow$ שיח/דשא (מסונן).

גובה ≤ 2 מ' $\leftarrow$ עץ (רדיוס צמרת = $0.4 \times \text{height}$).

סקירה מקצועית של הבעיה בעולם

נושא הניווט במרחב העירוני זוכה להתאסוסות רחבה במחקר ובפיתוח מערכות טכנולוגיות, בעיקר בהקשר של תכנון מסלולים להולכי רגל. לצד שיקולים של מרחק וזמן, קיימת התעניינות בגורמים סביבתיים המשפיעים על חוויית ההליכה, כגון טמפרטורה, קרינה סולארית ונוחות כללית. בעולם קיימות מספר גישות ופתרונות המתמודדים עם בעיות מקבילות או דומות, אם כי לא באופן משולב ומלא.

גישה ראשונה - מערכות ניווט מבוססות מפות עירוניות:

כגון יישומי ניווט להולכי רגל, המציעים מסלולים קצרים או מהירים. מערכות אלו מתבססות

על ייצוג גרפי של רשת הרחובות, כאשר כל קטע דרך משוקלל לפי מרחק או זמן הליכה.

יתרון: זמינות גבוהה, חישוב מהיר ויכולת ניווט מדויקת במרחב העירוני.

חסרון: היעדר התחשבות בגורמים סביבתיים כגון חשיפה לשמש או אזורים מוצלים.

גישה שנייה - מחקרי נוחות תרמית עירונית (Urban Thermal Comfort):

עוסקים בניתוח השפעת הסביבה הבנויה על תחושת החום והנוחות של הולכי רגל. מחקרים אלו משתמשים במודלים פיזיקליים של קרינה סולארית, טמפרטורה, רוח וצל, ומפיקים מדדים ומפות תרמיות של אזורים עירוניים.

יתרון: ניתוח מעמיק של השפעת הצל והמבנה העירוני על תנאי ההליכה.

חסרון: פתרונות אלו הם לרוב מחקריים או תכנוניים, ואינם מיועדים לשימוש אישי בזמן אמת.

גישה שלישית - מערכות סימולציה וניתוח צל עירוני:

נפוצות בתחומי האדריכלות והתכנון העירוני. מערכות אלו מחשבות את הטלת הצל של מבנים בהתאם לזווית השמש, לשעת היום ולעונות השנה.

יתרון: יכולת חישוב מדויקת של אזורי צל על בסיס נתונים גאומטריים.

חסרון: היעדר שילוב עם מנגנוני ניתוב דינמיים והיעדר התאמה לניווט אישי.

מקורות מידע על עצמים היוצרים צל בפתרונות הקיימים:

מאגרי מידע גיאוגרפיים (GIS): מיקום מבנים, קווי מתאר ונתוני גובה - נאספים על ידי רשויות מקומיות וגופים ציבוריים.

מודלים תלת - ממדיים של ערים: ייצוג מדויק של גובה וצורת מבנים לחישובי הצללה.

עיבוד תצלומי לוויין: שימוש באלגוריתמים של ראייה ממוחשבת לזיהוי מבנים ועצים.

מסקנה:

הפתרונות הקיימים בעולם מתמודדים כל אחד עם היבט חלקי בלבד - ניווט, ניתוח סביבתי או סימולציה צל - אך אינם משלבים בין התחומים באופן אחיד. הפרויקט נועד להשלים פער זה באמצעות שילוב חישוב שמש דינמי, נתוני מבנים ועצים, ואלגוריתם ניתוב - לכדי מערכת אחת.

נושאים רלוונטים במדעי המחשב בהקשר לבעיה

אלגוריתם דייקסטרה (Dijkstra's Algorithm):

דייקסטרה הוא אלגוריתם למציאת המסלול בעל העלות הנמוכה ביותר ממקור יחיד בגרף ממושקל. פיתח אותו אדסחר דייקסטרה ב-1959.

עיקרון הפעולה:

1. אתחול: $dist[\text{מקור}] = 0$, כל שאר הצמתים $= \infty$.
2. בכל צעד: בחר צומת עם מרחק מינימלי מתוך הלא - נחקרים.
3. הקלה (Relaxation): אם $dist[u] + w(u,v) < dist[v]$ אז $dist[v] \leftarrow dist[u] + w(u,v)$.
4. המשך עד הגעה ליעד.

מורכבות: זמן $O(E \log V)$, מקום $O(V)$.

מדוע דייקסטרה ולא A^* ?

A^* דורש heuristic מבוסס - מרחק, אך מסלול מוצל עשוי לסטות מהכיוון הישיר - ולכן heuristic כזה אינו admissible. דייקסטרה מבטיח אופטימליות גלובלית.

תור עדיפויות (Priority Queue):

מיושם כ- Binary Heap - הוצאת מינימום ב- $O(\log n)$, הכנסה ב- $O(\log n)$.

בפרויקט: PriorityQueue המובנה של C# (.NET 6+).

נוחת Haversine:

מחשבת מרחק על פני כדור הארץ בין שתי נקודות GPS:

$$d = 2R \times \arcsin(\sqrt{(\sin^2(\Delta\phi/2) + \cos(\phi_1) \times \cos(\phi_2) \times \sin^2(\Delta\lambda/2))})$$

$R = 6,371,000$ מטר. דיוק: 0.3%.

נוסחת Bearing (כיוונית):

כיוון מנקודה A לנקודה B:

$$\text{bearing} = \text{atan2}(\sin(\Delta\lambda) \times \cos(\phi_2), \cos(\phi_1) \times \sin(\phi_2) - \sin(\phi_1) \times \cos(\phi_2) \times \cos(\Delta\lambda))$$

תיאור פתרונות קיימים לבעיה

1. Google Maps / Waze:

יתרון: ניווט מדויק, נתונים מעודכנים.

חסרון: לא מתחשב בצל כלל. מציע רק מסלול קצר/מהיר.

2. Apple Maps:

יתרון: שילוב נתיבי הליכה מסומנים.

חסרון: אין התחשבות בגורמים סביבתיים.

3. WeatherRoute:

יתרון: מתחשב בתנאי מזג אוויר.

חסרון: אינו מחשב צל - רק מזג אוויר כללי.

מסקנה:

אין מוצר מסחרי זמין שמחשב מסלול הליכה מוצל על בסיס מיקום שמש, מבנים ועצים בזמן אמת.

ShadyWay ממלא פער זה.

ניתוח חלופות מערכתי

חלופות לאלגוריתם ניתוב

- A* (A-Star)

יתרון: מהיר יותר מדייקסטרה לחיפוש מנקודה לנקודה.

חסרון: מחייב heuristic מתאים. כאשר המשקולות כוללות צל, המרחק האווירי

אינו heuristic אמין כי המסלול המוצל עשוי לסטות מהכיוון הישיר.

מסקנה: נפסל ה- euristic אינו admissible בבעיה שלנו.

- Dijkstra ←← נבחרה

יתרון: מבטיח מסלול אופטימלי, $O(E \log V)$, עובד לכל משקולות לא-שליליות.

מסקנה: נבחרה- פשוט, אמין, ומבטיח אופטימליות.

חלופות לנתוני מבנים

- Google Open Buildings v3

מכיל מאות מיליוני מבנים גלובלית.

יתרון: כיסוי נרחב.

חסרון: אין נתוני גובה לישראל. ישראל אינה מכוסה היטב.

מסקנה: נפסלה - גובה המבנה קריטי לחישוב הצל.

- GHSL (Global Human Settlement Layer, JRC)

יתרון: כיסוי גלובלי עם גובה.

חסרון: רזולוציה של 100 מטר - גסה מדי לחישוב צל ברמת רחוב.

מסקנה: נפסלה - רזולוציה לא מספיקה.

- OSM (OpenStreetMap) ←← נבחרה
יתרון: אותו מקור נתונים המשמש לגרף הדרכים. חינמי. גובה זמין כ-height או levels.
אין צורך בקריאת API נוספת - המבנים מחולצים מה-XML שכבר קיים.
חסרון: לא כל המבנים מסומנים עם גובה.
מסקנה: נבחרה - יעיל, חינמי, ללא API נוסף.

חלופות לנתוני עצים

- חלופה א: OSM natural=tree
יתרון: נתונים ספציפיים לעצים, כולל גובה.
חסרון: ב-OSM ישראל- עצים ממופים באחוז קטן מאד. בדיקה בירושלים: 0 עצים ב-XML.
מסקנה: נפסלה- אין כיסוי מספיק.
- GEE ETH Global Canopy Height דרך ←← נבחרה
יתרון: רזולוציה 10 מטר, גלובלי, מדוד בלייזר ולווין, כיסוי ישראל מלא.
עצים < 2 מטר מזוהים אוטומטית. גובה אמיתי ולא הערכה.
חסרון: דורש הרשמה ל-GEE ו-Service Account.
מסקנה: נבחרה- הטובה ביותר מבחינת רזולוציה ואמינות.

חלופות לחישוב מיקום שמש

- נוסחאות פשוטות (הכרזה)
יתרון: קל למימוש.
חסרון: שגיאה של $2-5^\circ$ שמשפיעה על כיוון הצל.
מסקנה: נפסלה- לא מדויק מספיק.
- SPA (Solar Position Algorithm) - NREL
יתרון: דיוק $> 0.0003^\circ$ - מדויק ביותר.
חסרון: מורכבות גבוהה (מאות נוסחאות), קשה ליישום נכון.
מסקנה: נפסלה- מורכב מדי ללא תועלת מוספת.
- Meeus - Astronomical Algorithms ←← נבחרה
יתרון: דיוק $> 0.01^\circ$, 14 שלבי חישוב ניתנים לאימות, בסיס לרוב יישומי האסטרונומיה.
ניתן לאמת מול טבלאות ידועות. אין תלויות חיצוניות.
מסקנה: נבחרה - איזון מצוין בין דיוק ומורכבות.

חלופות לשפת תכנות

- Python (Flask/FastAPI)
יתרון: מהיר לפיתוח, ספריות גאוגרפיות רבות.
חסרון: ביצועים נמוכים מ-C# לחישובים מתמטיים כבדים.
מסקנה: נפסלה- #C מהיר יותר לחישובים.
- Java (Spring Boot)
יתרון: חזק ומוכן.
חסרון: verbose יותר, הגדרה ראשונית מורכבת יותר.
מסקנה: נפסלה- #C נלמד בקורס.

- #C עם ASP.NET Core ←← נבחרה
יתרון: מהיר, Onion Architecture, type-safe טבעי, EF Core לJWT, DB מובנה.
מסקנה: נבחרה- שפת הלימוד, ביצועים טובים, ecosystem עשיר.

תיאור החלופה הנבחרת + נימוקים

- **אלגוריתם דייקסטרה:**
נבחר בגלל אחריות לאופטימליות, פשטות יישוב, ועבודה עם משקולות לא-שליליות. עלות הקצה המשולבת (מרחק + צל) מאפשרת לאלגוריתם לאזן בין שני קריטריונים.
- **OSM למבנים:**
נבחר כי המבנים מגיעים בחינם עם אותה קריאת API של הדרכים. חיסכון של בקשת API נוספת ושמירה על זמן תגובה סביר. גובה נגזר ממספר קומות.
- **ETH/GEE לעצים:**
נבחר כי מספק נתוני גובה אמיתיים לכל מיקום גלובלי. נחוץ כי OSM ישראל חסר עצים.
- **Meeus לחישוב שמש:**
נבחר כי מספק דיוק מספיק (0.01°) ללא מורכבות מיותרת. ניתן לאמת, ובד"כ מיושם בכלים מוכרים.
- **Onion Architecture:**
נבחר כי מפריד בין לוגיקה עסקית לתשתית, מה שמקל על בדיקות ועתידי שינוי provider.

אפיון המערכת שהוגדרה / מוצעת

ניתוח דרישות המערכת

סביבת פיתוח:

- עמדת פיתוח: מחשב נייד / שולחני.
- מערכת הפעלה: Windows 11 Home / Pro.

חומרה:

- מעבד: Intel Core i5 ומעלה.
- זיכרון RAM: 8GB מינימום (מומלץ 16GB).
- אחסון: 500MB לפרויקט + 1GB ל-PostgreSQL.

מסד נתונים:

- PostgreSQL 15 + עם הרחבת PostGIS 3.

חיבור לרשת:

- נדרש חיבור לאינטרנט בזמן ריצה לקריאת OSM ו-GEE.
- Service Account מוגדר ב-Google Cloud עם גישה ל-GEE.

תוכנות:

- .NET 8 SDK
- Node.js 20
- Visual Studio 2022 / VS Code
- pgAdmin 4 (לניהול מסד הנתונים).
- Git

מודול המערכת

תחומים בהם המערכת עוסקת:

- חישוב מסלול הליכה מוצל בין שתי נקודות.
- ניהול משתמשים (הרשמה, התחברות).
- שמירת היסטוריית מסלולים.
- חישוב מיקום שמש בזמן אמת.
- הצגת המסלול על מפה אינטראקטיבית.

תחומים שהמערכת אינה עוסקת בהם:

- ניווט בזמן אמת (turn-by-turn).
- נתוני תחבורה ציבורית.
- תחזית מזג אוויר.

אפיון פונקציונלי

1. הרשמה- יצירת משתמש חדש עם אימייל, שם מלא, ת"ז וסיסמה.
2. התחברות- אימות וקבלת JWT Token.
3. חישוב מסלול- קלט: נקודות מוצא ויעד, שעה, העדפת צל; פלט: נתיב, מרחק, זמן, אחוז צל.
4. היסטוריה- שליפת כל המסלולים שהמשתמש ביקש.
5. הגדרת העדפה- בחירת shadow_preference בין 1.0 ל-2.0.

ביצועים עיקריים

ביצועי האלגוריתם המרכזי- מורכבות זמן:

1. בניית גרף מ-OSM: $N - O(N)$ מספר nodes ב-XML.
2. חישוב צל לכל קשת: $E - O(E \times 5 \times B)$ קשתות $B =$ מבנים/עצים בטווח 60 מ'.
3. אלגוריתם דייקסטרה: $E - O(E \log V)$ קשתות, $V =$ צמתים.
4. שחזור מסלול: $O(V)$.

זמני תגובה בפועל (אזור 1×1 ק"מ):

1. קריאת OSM + בניית גרף: ~ 1.5 שניות.
2. קריאת GEE (עצים): ~ 2.0 שניות.
3. חישוב צל לכל הקשתות: ~ 1.0 שניות.
4. ריצת דייקסטרה: > 0.1 שניות.

נפח נתונים:

1. גרף מרחב $1 \text{ ק"מ} \times 1 \text{ ק"מ} \approx 500-1000$ צמתים.
2. זיכרון שרת: $> 500 \text{ MB}$ בעומס רגיל.

אילוצים

- GEE API מחייב Service Account מוגדר מראש.
- OSM מחייב חיבור לאינטרנט (Overpass API).
- JWT תפוגה: 24 שעות.
- shadow_preference מוגבל ל-1.0-2.0.

תיאור הארכיטקטורה

הארכיטקטורה של הפתרון המוצע בפורמט של Top-Down level Design

המערכת בנויה לפי מודל רב-שכבתי (Layered Architecture) המאפשר הפרדה ברורה בין אחריות הלוגיקה, הנתונים והתצוגה. הגישה שנבחרה - Onion Architecture תכנן השכבות הפנימיות תחילה, כאשר הלוגיקה העסקית נמצאת במרכז ואינה תלויה בשכבות החיצוניות.

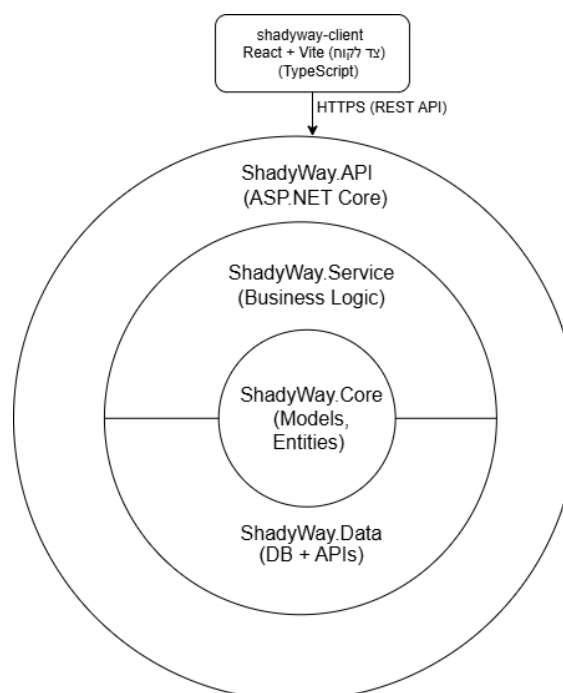
ShadyWay.Core שכבת הליבה (Domain) מכילה את המודלים הבסיסיים של המערכת: הנתונים User, RouteRequest, CalculatedRoute, SunPosition, RouteResult, BoundingBox, GraphEdge, GraphNode וכן את ישויות מסד שכבה זו ללא תלויות חיצוניות כלל היא הבסיס שעליו בנויות כל שאר השכבות.

ShadyWay.Infrastructure שכבת התשתית (Infrastructure) אחראית על כל תקשורת חיצונית: גישה ל-PostgreSQL באמצעות EF Core, קריאת נתוני OSM (XML parsing), גישה ל-GEE REST API לנתוני עצים, וקריאת נתוני מזג אוויר. שכבה זו מממשת את הממשקים שהוגדרו ב-Core. השם Infrastructure (ולא Data) נבחר בהתאם לעקרונות ארכיטקטורת הבצל (Onion Architecture), שבה שכבת הגישה לנתונים ולשירותים חיצוניים נקראת Infrastructure.

ShadyWay.Service שכבת השירות (Application) מרכז הלוגיקה העסקית. כולל את RouteService המנחה את כל תהליך חישוב המסלול, RouteHistoryService לשליפת היסטוריית מסלולים מה-DB, GraphBuilderService לבניית גרף הדרכים, ShadowCalculationService לחישוב אחוז הצל לכל קשת, DijkstraService להרצת האלגוריתם, ו-SunPositionService לחישוב מיקום השמש לפי Meeus.

ShadyWay.API שכבת ה-API (Presentation) שכבת הממשק בין הלקוח לשרת. כולל Controllers החושפים endpoints לפעולות: הרשמה, כניסה, חישוב מסלול, שליפת היסטוריה. כל endpoint מוגן ב-[Authorize] מלבד login/register.

shadyway-client שכבת התצוגה (React) ממשק משתמש דינמי הבנוי ב-React 18 ו-TypeScript, המאפשר הזנת נקודות מוצא ויעד, הצגת המסלול המוצל על מפה (MapLibre GL). וצפייה בהיסטוריית מסלולים.



תיאור הרכיבים בפתרון

מערכת הפרויקט בנויה מארכיטקטורת שרת-לקוח, וכוללת מספר רכיבים עיקריים:

שרת מסד הנתונים (Database Server) אחראי לאחסון הנתונים במבנה מסודר וניתן לשליפה. בפרויקט זה נעשה שימוש ב- PostgreSQL 15 עם הרחבת PostGIS, השומר את פרטי המשתמשים, בקשות המסלולים, והמסלולים המחוברים כולל הגיאומטריה המלאה (LineString).

שרת יישומים (Application Server) מרכז הלוגיקה העסקית. הוא מקבל את בקשת המסלול מהלקוח, מבצע את כל שלבי החישוב GEE, OSM, (שמש, צל, דייקסטרה) כותב נתונים למסד הנתונים, ומחזיר ללקוח את המסלול. השרת כתוב ב- C# ו- NET 8.

שרת תקשורת API מספק ממשק RESTful בין הלקוח לשרת, מבוסס ASP.NET Core. מקבל בקשות HTTPS מהלקוח (React), שולח אותן לשכבת השירות, ומחזיר את התשובות. שכבה זו גם מנהלת אימות, JWT אישורי גישה, ובקרת שגיאות.

לקוח (Client) ממשק המשתמש הבנוי ב- React ומאפשר בחירת נקודות על מפה או חיפוש כתובת/עיר, הגדרת העדפת צל, הצגת המסלול המוצל עם פרטיו (מרחק, זמן, אחוז צל), וצפייה בהיסטוריה.

תיאור תהליכים של מערכת ההפעלה

במהלך ריצת המערכת, מערכת ההפעלה ממלאת מספר תפקידים חיוניים המאפשרים את ביצוע תהליכי החישוב באופן תקין:

ראשית, מערכת ההפעלה אחראית על **ניהול חיבורי רשת** - פתיחת חיבורי HTTPS אל Overpass API לנתוני OSM ואל Google Earth Engine (לנתוני עצים), וסגירתם עם סיום הבקשה. בנוסף, היא מטפלת **בהקצאת זיכרון** עבור טעינת נתוני ה- XML, בניית הגרף ואחסון תוצאות החישוב הביניים.

תהליך נוסף כולל **ניהול תהליכי החישוב** - תזמון ביצוע אלגוריתם דייקסטרה, חישוב מיקום השמש, וחישוב אחוז הצל לכל קשת בגרף. לבסוף, מערכת ההפעלה מקצה **משאבי תקשורת** לשמירת הנתונים ב- PostgreSQL ולתקשורת בין רכיבי השרת לממשק המשתמש.

ארכיטקטורת רשת

המערכת כוללת שלושה ערוצי תקשורת ברשת:

- **לקוח ↔ API**: התקשורת בין ממשק המשתמש (React) לשרת (ASP.NET Core) מתבצעת דרך - HTTPS פרוטוקול מאובטח המצפין את כל הנתונים המועברים.
 - **API → OSM (Overpass)**: השרת שולח בקשות HTTPS לשרתי Overpass API כדי לשלוף נתוני מפה (דרכים ומבנים) לפי BoundingBox.
 - **API → Google Earth Engine**: השרת שולח בקשות HTTPS עם Bearer Token של Service Account לקבלת נתוני גובה עצים (ETH Canopy Height).
- כל התקשורת מתבצעת מצד השרת בלבד - הלקוח (React) אינו פונה ישירות לשירותים חיצוניים.

תיאור API בארכיטקטורה

ה-API הינו RESTful ובנוי ב- ASP.NET Core. הוא חושף את ה Endpoints - הבאים:

הגנה	תיאור	Endpoint	Method
ציבורי	יצירת משתמש חדש + קבלת JWT Token	/api/auth/register	POST
ציבורי	קבלת JWT Token	/api/auth/login	POST
[Authorize]	חישוב מסלול מוצל	/api/route	POST
[Authorize]	שליפת היסטוריית מסלולים	/api/history	GET

כל endpoint מוגן ב- [Authorize] מלבד login/register בקשה ללא Token תקין מחזירה שגיאה 401. הנתונים מועברים בפורמט JSON בכל הבקשות והתשובות.

תיאור פרוטוקולי התקשורת

בפרויקט נעשה שימוש בפרוטוקולי תקשורת מודרניים וסטנדרטיים:

HTTPS פרוטוקול התקשורת המרכזי בין ממשק המשתמש (React) לבין שרת ה-API. הבקשות נשלחות בפרוטוקול HTTPS.

REST API הממשק בין הלקוח לשרת מבוסס על RESTful API. השרת חושף endpoints לפעולות כמו חישוב מסלול, הרשמה וכניסה. הנתונים מועברים בפורמט JSON.

Bearer Token כל בקשה מוגנת כוללת כותרת Authorization: Bearer <jwt_token> המאפשרת לשרת לאמת את זהות המשתמש.

HTTPS עם Service Account התקשורת מול Google Earth Engine מתבצעת עם אימות של Service Account של Google Cloud.

שרת- לקוח

הפרויקט מתבסס על ארכיטקטורת שרת- לקוח (Client- Server Model) המאפשרת הפרדה ברורה בין ממשק המשתמש לבין הלוגיקה העסקית ואחסון הנתונים.

הלקוח (Client) המבוסס על React מהווה את שכבת ההצגה ומספק ממשק משתמש אינטואיטיבי לבחירת מסלול, הגדרת העדפות וצפייה בתוצאות.

השרת (Server) מורכב משלוש שכבות:

Web API - שער הכניסה של המערכת, מקבל בקשות HTTPS מהלקוח ומעביר אותן לשכבת הלוגיקה. אחראי על ניהול הרשאות, אימות JWT ובקרת שגיאות.

שרת היישומים - מממש את הלוגיקה העסקית: בניית גרף, חישוב צל, ריצת דייקסטרה. כתוב ב- C# תחת .NET 8.

שרת מסד הנתונים - מבוסס על PostgreSQL+PostGIS, שומר את נתוני המשתמשים, הבקשות והמסלולים המחושבים.

מודל זה מבטיח שהמערכת תישאר יציבה, ניתנת לתחזוקה ולהרחבה, תוך הפרדה ברורה בין שכבות הקוד.

תיאור תהליכי אבטחת מידע במערכת

תיאור התקיפה או ההגנה

האיומים המרכזיים עליהם המערכת מגינה:

- גישה לא מורשית לנתוני משתמשים אחרים.
- גניבת סיסמאות ממסד הנתונים.
- יירוט נתונים בתקשורת.
- קלט זדוני מהמשתמש (Injection, ערכים לא תקינים).
- גישה ממקורות לא מורשים (Cross-Origin attacks).
- חשיפת מפתחות סודיים בקוד המקור.

הגנות שמומשו בפרויקט:

1. הצפנת סיסמאות עם BCrypt:

הסיסמה לא נשמרת בטקסט רגיל. בהרשמה:
`passwordHash = BCrypt.HashPassword(password)`
 Hash כולל salt רנדומלי ולא ניתן לשחזור. בכניסה:
`BCrypt.Verify(inputPassword, storedHash)`

כך גם אם מסד הנתונים נחשף הסיסמאות אינן קריאות.

2. JWT Authentication:

- אחרי כניסה מוצלחת, השרת מנפיק JWT חתום עם HMAC-SHA256:
- מפתח סודי מאוחסן ב-User Secrets (לא בקוד)
- Claims: userId, email, fullName
- תפוגה: 24 שעות

כל ה- endpoint שמוגנים עם [Authorize] בקשה ללא Token תחזיר 401.

3. ולידציה של קלט:

- ב-RegisterDto, ASP.NET Core מאמת אוטומטית לפני כניסה לקוד:
- תעודת זהות: `RegularExpression(^d{9}$)`
- אימייל: `[EmailAddress]`
- סיסמה: `[MinLength(8)]`
- `shadow_preference: [Range(1.0, 2.0)]`

קלט שאינו עומד בתנאים מחזיר 400 Bad Request.

4. הרשאות:

כל משתמש יכול לגשת רק לנתונים שלו. `HistoryController`:
`var userId = User.FindFirstValue(ClaimTypes.NameIdentifier)`
`user_id == userId` עם rows רק

גישה לנתונים של משתמש אחר בלתי אפשרית.

5. HTTPS עם הפניה אוטומטית:

המערכת מוגדרת עם `UseHttpsRedirection` - כל בקשה ב-HTTP מופנית אוטומטית ל-HTTPS.

התקשורת בין הלקוח לשרת מוצפנת לחלוטין דרך TLS.

6. CORS Policy:

הוגדרה מדיניות CORS המגבילה גישה ל-API רק ממקורות מורשים:
`policy.WithOrigins("https://localhost:5173", "https://localhost:3000")`
 בקשות ממקורות אחרים נדחות אוטומטית על ידי הדפדפן.

7. הגנה על קבצים רגישים:

- `service-account-key.json` (GEE): מוסף ל-`.gitignore`.
- `JWT Key`: מאוחסן ב-`User Secrets`, לא ב-`appsettings.json`.

תיאור הצפנות

- `BCrypt`: אלגוריתם `Adaptive hashing` לסיסמאות - כולל `salt` רנדומלי ואין אפשרות לשחזר את הסיסמה המקורית.
- `HMAC-SHA256`: לחתימת `JWT Token` - מבטיח שהטוקן לא שונה.
- `HTTPS/TLS`: הצפנת כל התקשורת בין הלקוח לשרת - מונע יירוט נתונים.

למידת מכונה

אין התייחסות בפרויקט זה.

תיאור התוכנה

פירוט API

ממשק ה-API של `ShadyWay` מורכב מ-4 נקודות קצה (Endpoints) המחולקות לשני תחומים: אימות משתמשים (Auth) וניווט (Route/History).
 כל הבקשות והתשובות מועברות בפורמט `JSON` מעל פרוטוקול `HTTPS`.
 נקודות הקצה המסומנות "מוגן" דורשות כותרת `Authorization: Bearer <token>`.

— אימות —

`POST /api/auth/register` [ציבורי]

תיאור: הרשמת משתמש חדש למערכת.

גוף בקשה: `{ "fullName": "תמר שימל", "identityCard": "215949371" }`

תשובה מוצלחת (200): `{ "shadowPreference": 1.5, "password": "Aa123456", "email": "tamar@example.com", "...token": "eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9" }`

שגיאות אפשריות: `400 Bad Request` (קלט חסר / ת.ז. כבר קיימת)

`POST /api/auth/login` [ציבורי]

תיאור: כניסת משתמש קיים וקבלת טוקן `JWT`.

גוף בקשה: `{ "fullName": "תמר שימל", "email": "tamar@example.com", "password": "Aa123456" }`

תשובה מוצלחת (200): `{ "shadowPreference": 1.5, "email": "tamar@example.com", "fullName": "תמר שימל", "...token": "eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9" }`

שגיאות אפשריות: `401 Unauthorized` (פרטים שגויים)

— ניווט —

`POST /api/route` [מוגן - JWT]

תיאור: חישוב מסלול הולכי הרגל המוצל ביותר בין שתי נקודות. השרת בונה גרף, מחשב צל לכל קשת, ומריץ אלגוריתם `Dijkstra`.

גוף בקשה: [{ "startLongitude": 35.2137, "startLatitude": 31.7683, "shadowPreference": 1.5, "endLongitude": 35.2200, "endLatitude": 31.7750, "utcDateTime": "2025-07-15T10:00:00Z" }]
 שדה shadowPreference: 1.0 = מסלול קצר ביותר, 2.0 = מוצל ביותר.
 שדה utcDateTime: אם לא נשלח, השרת משתמש בשעה הנוכחית.
 תשובה מוצלחת (200): [{ "totalDistanceMeters": 450.5, "found": true, "estimatedWalkingTimeMinutes": 5.4, "averageShadowPercentage": 67.3, "path": [{ "latitude": 31.7683, "longitude": 35.2137 }, { "latitude": 31.7695, "longitude": 35.2150 }, ...] }]
 שגיאות אפשריות: 401 (טוקן חסר/פג), 404 (מסלול לא נמצא), 500 (שגיאת שרת).

GET /api/history [מוגן - JWT]

תיאור: שליפת 20 בקשות המסלול האחרונות של המשתמש המחובר.
 הנתונים נשלפים ממסד הנתונים ומאפשרים חישוב מחדש של מסלול קודם בלחיצה אחת - ללא הקלדת נקודת מוצא ויעד מחדש.
 (שימוש אופייני: משתמש שהולך כל יום לאותו יעד יכול לחשב מחדש את המסלול עם מיקום השמש הנוכחי).
 תשובה מוצלחת (200): [{ "requestId": 1, "sourceAddress": "31.7683,35.2137", "requestTime": "2025-07-15T10:05:00Z", "destAddress": "31.7750,35.2200", "shadowPercentage": 67.3, "totalDistanceMeters": 450.5 }]
 הערה: שדות sourceAddress ו-destAddress שומרים את הקואורדינטות בפורמט "latitude,longitude". הלקוח מפרק אותן ומציג אותן למשתמש, ובלחיצה על "חשב מחדש עם השמש הנוכחית" שולח POST /api/route עם אותן הנקודות ועם שעת UTC הנוכחית.

סביבת עבודה

IDE סביבת פיתוח:

- **Visual Studio 2022** שימש לכתובת קוד ה- C# השרת, הלוגיקה, ה-API.
- **VS Code** שימש לכתובת קוד ה- React מממשק המשתמש.

Database IDE ניהול מסד הנתונים:

- **pgAdmin 4** תוכנה גרפית לניהול PostgreSQL. דרכה ניתן לראות טבלאות, להריץ שאילתות SQL, ולבדוק שהנתונים נשמרו נכון.

API Testing בדיקת ה-API:

- **Swagger UI** ממשק ויזואלי שנוצר אוטומטית מהקוד. מאפשר לשלוח בקשות HTTPS לכל endpoint ולראות תשובות, ישירות מהדפדפן בכתובת <https://localhost:7168/swagger>

Runtime סביבת הרצה:

- **NET 8** הפלטפורמה שמריצה את קוד C# של השרת.
- **Node.js 20** הפלטפורמה שמריצה את Vite כלי ה- build של React.

שפות תכנות

הפרויקט ממומש בשלוש שפות תכנות:

1. C# - צד השרת (Backend) שפת הפיתוח הראשית של הפרויקט. משמשת לכתובת כל לוגיקת הניווט, חישוב הצל, ובניית הגרף.
2. TypeScript / TSX - צד הלקוח (Frontend) TypeScript היא JavaScript עם הגדרות טיפוסים סטטיים, מה שמאפשר גילוי שגיאות בזמן פיתוח ולא רק בזמן ריצה. TSX (TypeScript + JSX) משמש לכתובת רכיבי React עם תמיכה מלאה בטיפוסים.
3. SQL - מסד הנתונים שאילתות SQL (דרך EF Core) משמשות לשמירה ושלפית נתוני משתמשים, בקשות מסלול, והיסטוריה. שאילתות PostgreSQL גיאוגרפיות מאפשרות שמירת קואורדינטות ומסלולים.

ניתוח ותרשים UML / Use cases של המערכת המוצעת

תיאור ה-UC העיקריים של המערכת

האלגוריתם הראשי:

האלגוריתם הראשי של ShadyWay הוא חישוב מסלול הולכי הרגל המוצל ביותר בין שתי נקודות גיאוגרפיות. להלן שלבי האלגוריתם בסדר ביצוע:

שלב 1 - קבלת קלט:

קבלת נקודת מוצא (lat/lon), נקודת יעד (lat/lon), שעת UTC, והעדפת צל (1.0 = מסלול קצר, 2.0 = מסלול מוצל).

שלב 2 - חישוב BoundingBox:

$BoundingBox = (min_lat - pad, min_lon - pad, max_lat + pad, max_lon + pad)$
כאשר $pad = 15\%$ מגודל האזור בין שתי הנקודות.
מטרת הריפוד: לאפשר ניתוב עוקף שיוצא מהקו הישר בין A ל-B.

שלב 3 - קריאת OpenStreetMap (OSM):

קריאה אחת ל-Overpass API עם ה-BoundingBox.
מה-XML המתקבל: בניית גרף כבישים (nodes + edges) + חילוץ מבנים.

שלב 4 - קריאת Google Earth Engine (GEE):

קריאה ל-REST API לקבלת רשימת עצים עם קואורדינטות וגובה כופה.

שלב 5 - חישוב מיקום השמש:

אלגוריתם Meeus (14 שלבים) מחשב אזימוט וזווית גובה של השמש לפי lat, lon, UTC - בדיוק > 0.01 מעלות.

שלב 6 - חישוב אחוז צל לכל קשת בגרף:

לכל קשת (קטע רחוב):

א. 5 נקודות דגימה שוות לאורך הקטע.

ב. לכל נקודת דגימה - בדיקת כל מבנה ועץ בטווח 60 מטר.

ג. חישוב אם הגוף מטיל צל על הנקודה לפי גיאומטריה.

ד. ממוצע אחוז הצל $\times$ מקדם עננות (מ-OpenWeatherMap).

שלב 7 - אלגוריתם Dijkstra:

עלות קשת = $(1 + (1 - \text{shadow}/100) \times (\text{preference} - 1)) \times \text{distance}$
 בעדיפות 1.0 העלות = מרחק בלבד.
 בעדיפות 2.0 קשתות שטופות שמש "יקרות" יותר, ומסלול מוצל מועדף.

שלב 8 - שמירה ב-DB:

שמירת הבקשה (נקודות, שעה, העדפה) והמסלול (מקטעים, % צל)
 בטבלאות RouteRequests + CalculatedRoutes ב-PostgreSQL.

שלב 9 - החזרת תוצאה:

מחזיר: מסלול (רשימת קואורדינטות), מרחק כולל, זמן הליכה משוער, ואחוז צל ממוצע.

אלגוריתם משני - חישוב מיקום שמש (Meeus):

אלגוריתם 14 שלבים לחישוב אסטרונומי מדויק של מיקום השמש.
 קלט: lat, lon, UTC DateTime.
 פלט: Azimuth (כיוון השמש 0-360°), Elevation (גובה מעל האופק).
 שימוש: כל קשת שבה הצל של מבנה/עץ נופל - נחשבת לנקודת הדגימה.

הצגת מקרה (Use Case) עבור כל הפונקציות העיקריות בפרויקט

UC1 - הרשמה למערכת	
יצירת חשבון משתמש חדש.	תיאור
משתמש לא מחובר.	שחקן
המשתמש אינו רשום במערכת.	תנאי מקדים
לחיצה על כפתור "הרשמה".	הזנק
1. הזנת ת.ז., שם מלא, אימייל, סיסמה, העדפת צל. 2. ולידציה בצד הלקוח. 3. שליחת POST /api/auth/register. 4. BCrypt.Hash (סיסמה), שמירה ב-DB. 5. יצירת JWT Token (HMAC-SHA256 120 דקות תוקף) והחזרתו ללקוח.	מסלול עיקרי
משתמש נוצר במערכת, JWT Token ישלח, משתמש מחובר.	תנאי סיום
פעם אחת בלבד לכל משתמש.	תדירות

UC2 - כניסה למערכת	
אימות זהות וקבלת Token.	תיאור
משתמש רשום.	שחקן
המשתמש קיים במערכת.	תנאי מקדים
לחיצה על כפתור "כניסה".	הזנק
1. הזנת שם מלא, אימייל, וסיסמה. 2. ולידציה בצד הלקוח. 3. שליחת POST /api/auth/login. 4. BCrypt.Verify (סיסמה, hash שמור). 5. יצירת JWT Token (HMAC-SHA256 120 דקות תוקף) והחזרתו ללקוח.	מסלול עיקרי
JWT Token נשמר אצל הלקוח, המשתמש עובר למפה.	תנאי סיום
כל פעם שמשתמש נכנס למערכת.	תדירות

UC3 - חישוב מסלול מוצל	
תיאור	מציאת מסלול ההליכה מוצל בין נקודות A ו-B.
שחקן	משתמש מחובר.
תנאי מקדים	המשתמש מחובר (JWT תקין), שתי נקודות נבחרו.
הזנק	לחיצה על "חשב מסלול מוצל".
מסלול עיקרי	1. קריאת OSM, בניית גרף וקבלת מבנים. 2. קריאת GEE, קבלת עצים. 3. חישוב מיקום שמש (Meeus). 4. חישוב % צל לכל קשת. 5. Dijkstra עם עלות משולבת. 6. שמירה ב-DB, החזרת מסלול.
תנאי סיום	מסלול מוצג על המפה עם מרחק, זמן ו-% צל.
תדירות	עיקרית - פעמים רבות.

UC5 - חישוב מחדש ממסלול קודם	
תיאור	שימוש במסלול שמור לחישוב חדש עם שמש עדכנית.
שחקן	משתמש מחובר.
תנאי מקדים	קיים לפחות מסלול אחד בהיסטוריה (UC4 הושלם).
הזנק	לחיצה על "חשב מחדש עם השמש הנוכחית".
מסלול עיקרי	1. חילוץ קואורדינטות מ-sourceAddress/destAddress. 2. קריאה ל-runCalculation עם UTC נוכחי. 3. ביצוע UC3 מחדש עם אותן נקודות ושעה חדשה.
תנאי סיום	מסלול מחושב מחדש עם מיקום שמש עדכני.
תדירות	בינונית - שימוש יומי חוזר לאותו מסלול.

UC6 - יציאה מהמערכת	
תיאור	ניתוק המשתמש מהמערכת.
שחקן	משתמש מחובר.
תנאי מקדים	המשתמש מחובר.
הזנק	לחיצה על כפתור "יציאה".
מסלול עיקרי	1. מחיקת ה-JWT Token מצד הלקוח (localStorage). 2. מעבר למסך הכניסה.
תנאי סיום	Token נמחק, המשתמש אינו מחובר.
תדירות	בכל פעם שמשתמש יוצא מהמערכת.

מבני נתונים בהם השתמש (רשימה, מחסנית, תור וכו')

Dictionary<long, GraphNode> - ייצוג הגרף הראשי:

מפתח = OSM Node ID (long), ערך = אובייקט GraphNode.

נבחר כי גישה לצומת לפי מזהה היא $O(1)$ - קריטי ב-Dijkstra.

שמבצע עשרות אלפי גישות לצמתים במהלך ריצה.

List<GraphEdge> - בתוך כל GraphNode קשתות יוצאות:

רשימה של כל הכבישים היוצאים מכל צומת.

נבחרה רשימה (ולא מערך) כי מספר הקשתות לצומת אינו ידוע מראש ומשתנה (צומת-T יש לו 3, כיכר יש לה 4+).

PriorityQueue<long, double> - תור עדיפויות ל-Dijkstra:
מבנה הנתונים המתאים ביותר ל-Dijkstra - תמיד מוציא את הצומת עם עלות מינימלית ב- $O(\log N)$.
בדוק ב-.NET 8+ כמחלקה מובנית (System.Collections.Generic).
חלופה (מיון מחדש בכל איטרציה) הייתה $O(V^2)$ - בלתי אפשרי לגרפי עיר.

Dictionary<long, double> dist[] - מרחקים מינימליים:
שמירת עלות הגעה מינימלית לכל צומת.
גישה $O(1)$ לכל עדכון עלות - נדרש בכל צעד של Dijkstra.

Dictionary<long, long> prev[] - שחזור מסלול:
לכל צומת - מי הקודם שלו במסלול האופטימלי.
מאפשר שחזור המסלול הסופי ב- $O(V)$ הילוך לאחור מהיעד למוצא.

List<BuildingInfo> ו-**List<TreeInfo>**:
רשימות נוצרות פעם אחת מה-API, ומועברות לכל 5 נקודות הדגימה.
רשימה מספיקה כי הגישה היא עיבוד סדרתי (לכל מבנה - בדיקה).

חישוב יעילות האלגוריתמים

V = מספר צמתים בגרף, E = מספר קשתות, B = מבנים + עצים בטווח 60 מ'.
בניית גרף מ-OSM $O(N)$.
 N = מספר nodes ב-XML של OSM - מעבר יחיד על הנתונים.

חישוב צל לכל קשתות הגרף: $O(E \times 5 \times B)$
לכל קשת: 5 נקודות דגימה $\times$ בדיקת B עצמים.
ב-BoundingBox של 1 ק"מ: $2,000,000 \approx 2,000, B \approx 200, E \approx 2,000$ פעולות.

אלגוריתם Dijkstra: $O((V + E) \times \log V)$
עם $\log V$ - PriorityQueue לכל הוצאה מהתור.
ב-BoundingBox של 1 ק"מ: $2,000, E \approx 2,000, V \approx 1,000$ מהיר מאוד.

שחזור מסלול: $O(V)$
מעבר לאחור על מערך $prev[]$ מהיעד למוצא.

חישוב מיקום שמש (Meeus): $O(1)$
14 פעולות חישוב מתמטיות קבועות - ללא לולאות.
סה"כ המורכבות הדומיננטית: $O(E \times B)$ - חישוב הצל.

הקשרים בין היחידות השונות

1. **ממשק המשתמש (Client / React Frontend)**
ממשק המשתמש הוא נקודת הכניסה של המשתמש למערכת. הוא מאפשר לבצע פעולות כמו הרשמה, כניסה, בחירת נקודות על המפה/בחירת עיר וצפייה בהיסטוריה.
הממשק שולח בקשות דרך פרוטוקול HTTPS לשרת היישומים ומציג את התוצאות המתקבלות - מסלול מוצל המצויר על גבי מפה אינטראקטיבית.

2. **שרת היישומים (Application Server / ASP.NET Core API)**
שרת זה הוא ליבת המערכת. הוא מקבל בקשות מהממשק, מאמת את זהות המשתמש באמצעות JWT Token, ומעביר את הבקשה לשכבת הלוגיקה.

לאחר קבלת התוצאה מהשירותים הפנימיים, הוא מחזיר את המסלול ללקוח בפורמט JSON. בנוסף, הוא שומר את כל הבקשות והתוצאות במסד הנתונים לצורך היסטוריה.

3. שירות בניית גרף (GraphBuilderService)

שירות זה אחראי על שליפת נתוני המפה מ-OpenStreetMap (Overpass API) ובניית גרף הכבישים. הוא מקבל BoundingBox, שולח בקשת XML ל-OSM, ומחזיר מילון של צמתים וקשתות המייצגים את הרחובות באזור - יחד עם רשימת המבנים לצורך חישוב הצל.

4. שירות חישוב צל (ShadowCalculationService)

שירות זה מחשב את אחוז הצל לכל קשת בגרף. הוא פועל מול שלוש יחידות נוספות: SunPositionService לקבלת מיקום השמש, GoogleEarthClient לקבלת גובה עצים מ-Google Earth Engine, ו-WeatherClient לקבלת מקדם עננות מ-OpenWeatherMap. לכל קשת הוא בודק 5 נקודות דגימה ומחשב ממוצע צל.

5. שירות מיקום השמש (SunPositionService)

שירות זה מחשב את מיקום השמש לפי אלגוריתם 14 - Meeus שלבים מתמטיים, ללא קריאות לשירות חיצוני. הוא מקבל קואורדינטות ושעת UTC ומחזיר אזימוט וזווית גובה בדיוק של פחות מ-0.01 מעלות. תוצאתו משמשת את ShadowCalculationService לכל חישוב צל.

6. שירות הניווט (DijkstraService)

שירות זה מקבל את הגרף לאחר שחושב לו אחוז הצל, ומוצא את המסלול האופטימלי בין נקודת המוצא לנקודת היעד. הוא מפעיל את אלגוריתם Dijkstra עם עלות משולבת של מרחק וצל בהתאם להעדפת המשתמש, ומחזיר את רשימת הצמתים, המרחק הכולל ואחוז הצל הממוצע.

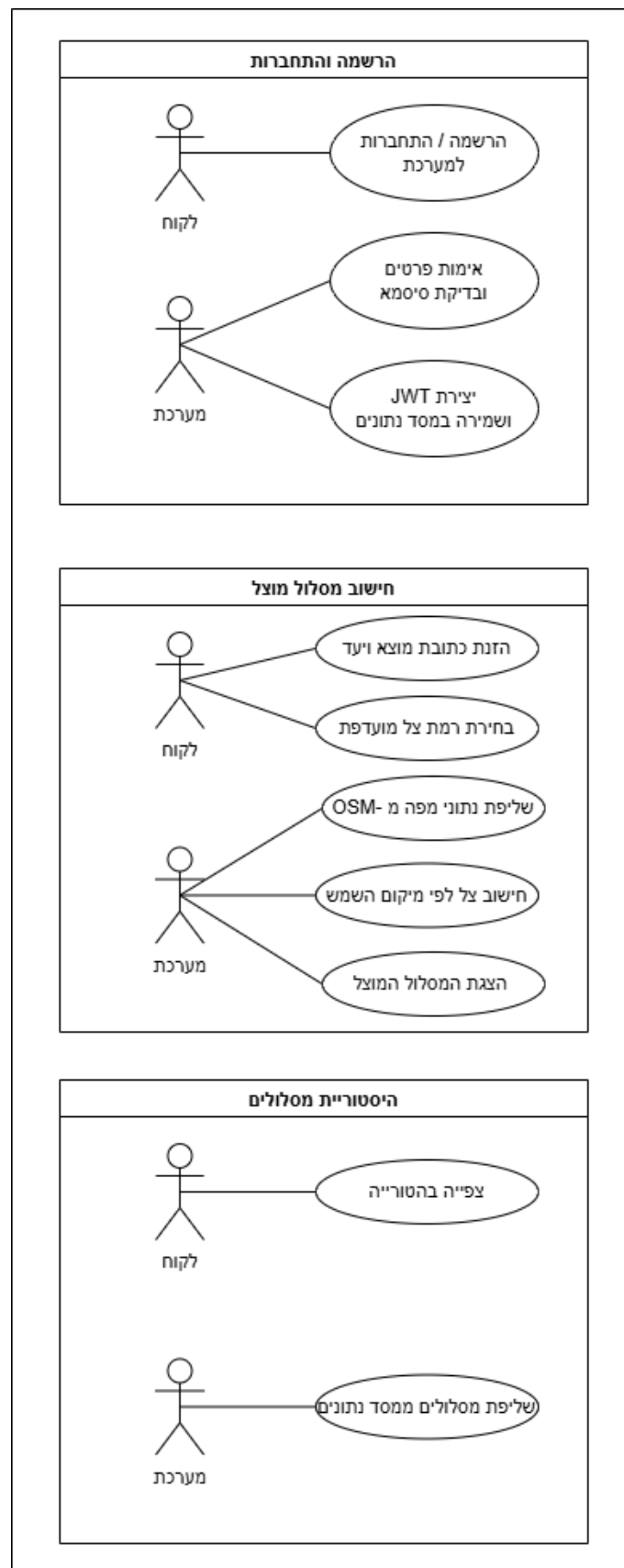
7. מסד הנתונים (Database Server / PostgreSQL + PostGIS)

מסד הנתונים שומר את פרטי המשתמשים, בקשות המסלול והמסלולים שחושבו. שרת היישומים מבצע קריאות וכתובות למסד באמצעות EF Core. הרחבת PostGIS מאפשרת שמירה ועיבוד של נתונים גיאוגרפיים. הנתונים משמשים להצגת היסטוריית המסלולים ולחישוב מחדש.

עץ מודולים



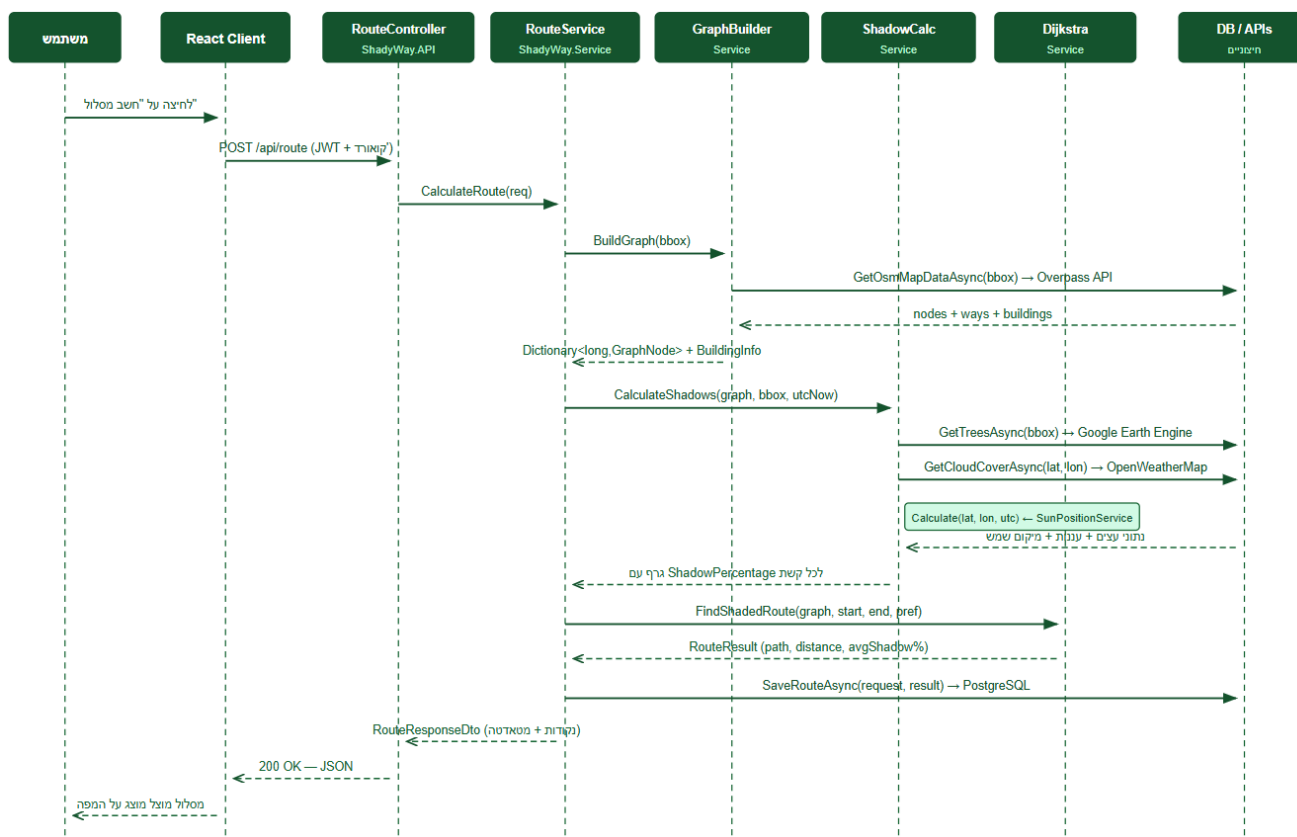
Use Case Diagram



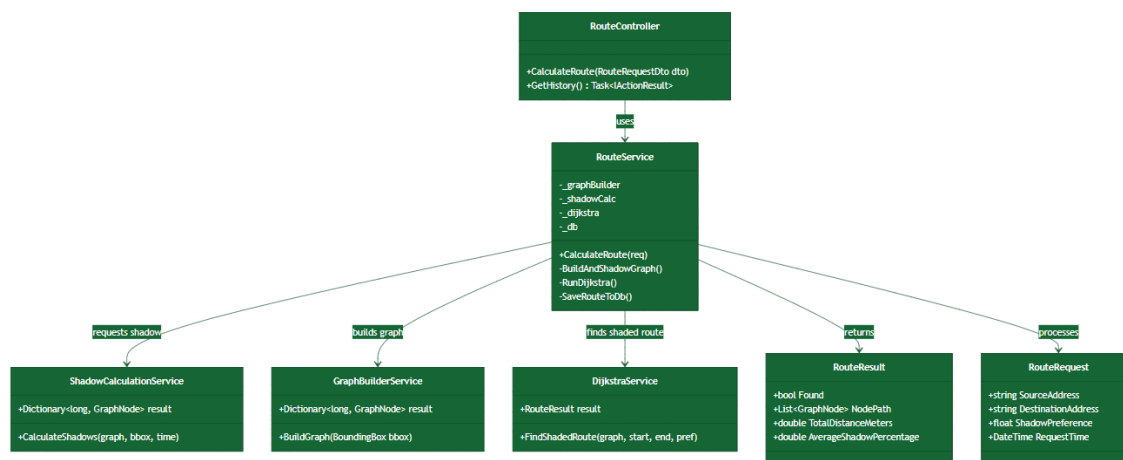
רשימת Use Cases

מזהה	שם ה- Use Case	שחקן	תיאור
UC1	הרשמה למערכת	משתמש לא מחובר	יצירת חשבון משתמש חדש
UC2	כניסה למערכת	משתמש רשום	אימות זהות וקבלת Token
UC3	חישוב מסלול מוצל	משתמש מחובר	מציאת מסלול הליכה מוצל בין נקודות A ו-B
UC4	צפייה בהיסטוריה	משתמש מחובר	הצגת 20 המסלולים האחרונים שהמשתמש ביקש
UC5	חישוב מחדש ממסלול קודם	משתמש מחובר	שימוש במסלול שמור לחישוב חדש עם שמש עדכנית
UC6	יציאה ממערכת	משתמש מחובר	ניתוק המשתמש מהמערכת

UML



Design class Diagram



תרשים מחלקות

להלן הצגת המחלקות לפי שכבה:

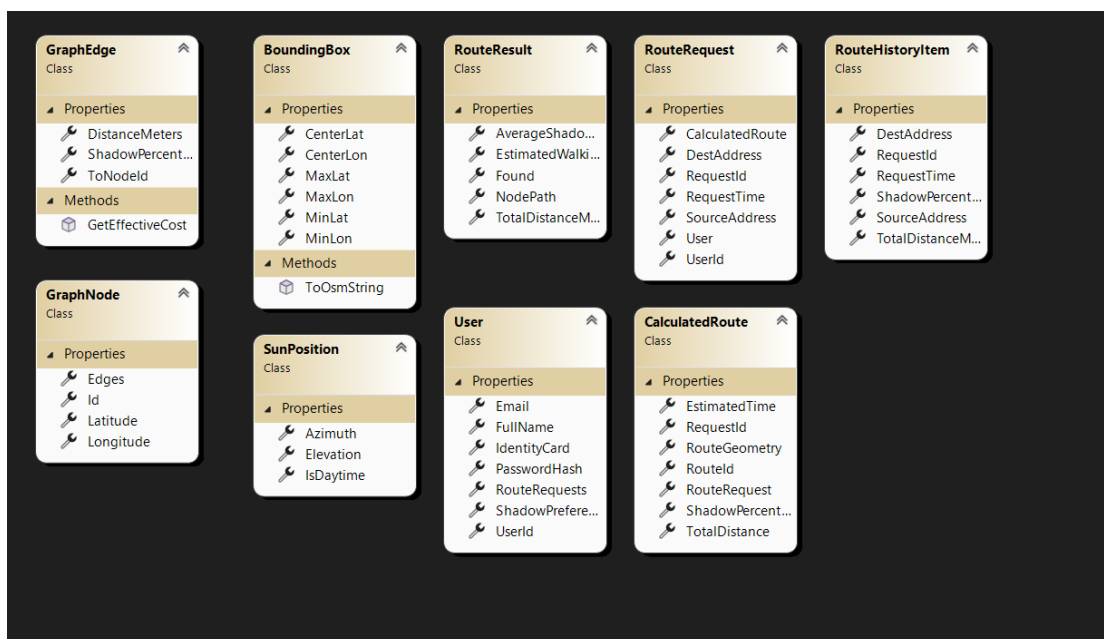
שכבת Core/Models - מודלים ואובייקטי הליבה

מחלקות אלו אינן תלויות בשום שכבה אחרת - הן מייצגות את ה"עולם" של הבעיה: צמתים, קשתות, מיקום שמש, ומשתמשים.

על פי עקרון Onion Architecture, הלוגיקה הפנימית ביותר (Models) לא תלויה בשרת, בממשק, ולא במסד הנתונים.

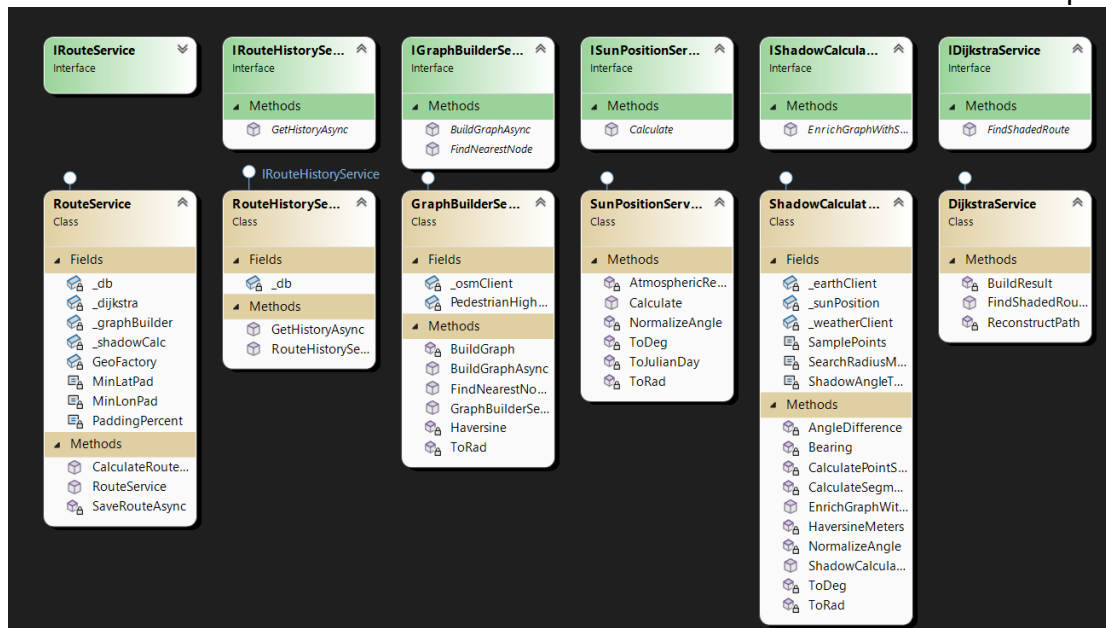
ניתן לחלק את המחלקות בשכבה זו לשלוש קבוצות:

- מחלקות גרף - GraphNode, GraphEdge
- מחלקות עזר לחישוב - BoundingBox, SunPosition, RouteResult
- ישויות מסד הנתונים - User, RouteRequest, CalculatedRoute
- מודלי תוצאה - RouteHistoryItem (Read Models)



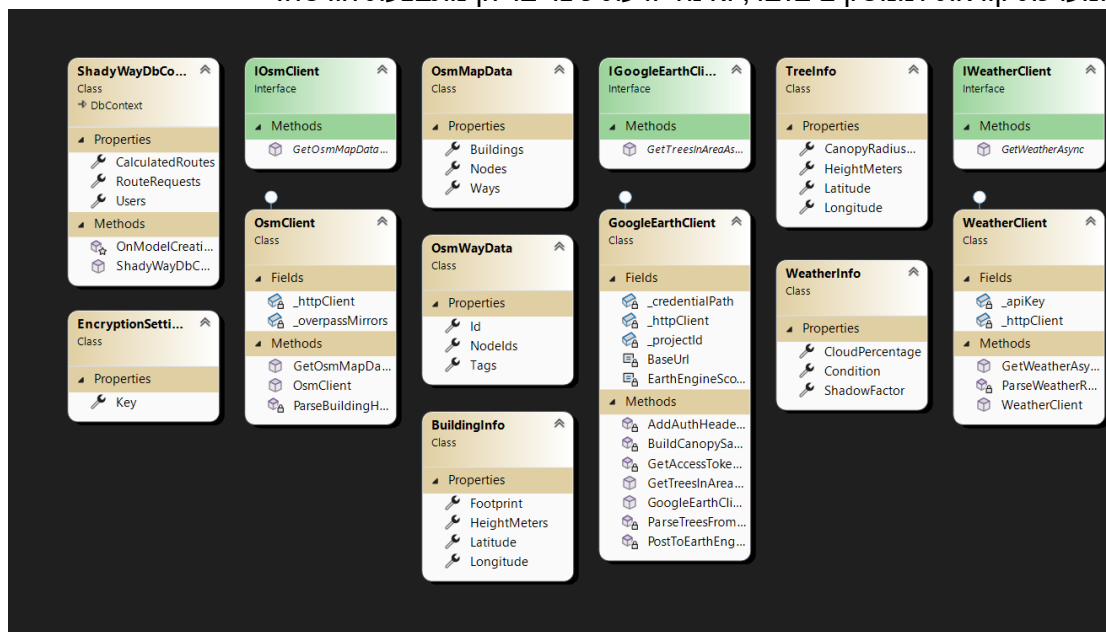
שכבת Service - לוגיקה עסקית

שכבת Service מכילה את כל "השכל" של המערכת – הכללים העסקיים, חישובי הצל, בניית הגרף ואלגוריתם הניתוב.



שכבת Infrastructure - לקוחות חיצוניים ומסד נתונים

שכבה זו "מסתירה" את הפרטים הטכניים של גישה לשירותים חיצוניים ולמסד הנתונים - שאר המערכת קוראת לממשקים בלבד, ואינה יודעת כיצד בדיוק מתבצעת הגישה.



תיאור המחלקות המוצעות - מהכלל אל הפרט

להלן תיאור כל יחידה בפרויקט.
עבור כל יחידה מפורטים: תפקידה, קלטיה (ומאיפה מגיעים) ופלטיה (ולאן הולכים).
בסוף הסעיף מוצגת זרימת המידע המלאה בין היחידות.
הערה: קלט יכול להגיע מהמשתמש, מהרשת (API חיצוני), ממסד הנתונים, או מיחידה אחרת.
פלט יכול להישלח למשתמש, לרשת, למסד הנתונים, או ליחידה אחרת.

שכבת Presentation (צד לקוח - React)

1. AuthForm:

תפקיד: מציגה טופס הרשמה/כניסה ושולחת פרטי משתמש לשרת.
קלטים:
מהמשתמש: שם מלא, תעודת זהות, אימייל, סיסמה, העדפת צל.
פלטים:
לרשת (POST /api/auth/register או login): פרטי ההרשמה/כניסה.
ליחידה App: JWT Token + שם משתמש לאחר אימות מוצלח.

2. RouteForm:

תפקיד: פאנל שליטה - בחירת נקודות (חיפוש כתובת או לחיצה על מפה), הגדרות, הפעלת חישוב.
קלטים:
מהמשתמש: הקלדת כתובת בשדה חיפוש/ לחיצה על כפתורי מפה, הזזת מחוון צל, בחירת שעה.
מיחידה App: תוצאת המסלול המחושב, היסטוריה, שם משתמש.
פלטים:
ליחידה App: בקשת חישוב מסלול (נקודות + הגדרות).
למשתמש: הצגת תוצאות (מרחק, זמן, % צל).

3. MapView:

תפקיד: מציגה מפה אינטראקטיבית ומאפשרת בחירת נקודות בלחיצה.
קלטים:
מהמשתמש: לחיצה על המפה לבחירת נקודת מוצא / יעד.
מיחידה App: רצף קואורדינטות המסלול לציון.
פלטים:
ליחידה App: קואורדינטות הנקודה שנבחרה.
למשתמש: הצגת המסלול כקו ירוק על המפה.

4. RouteHistory:

תפקיד: מציגה רשימת מסלולים קודמים ומאפשרת חישוב מחדש.
קלטים:
מיחידה RouteForm: רשימת ההיסטוריה (עד 20 מסלולים).
מהמשתמש: לחיצה על "חשב מחדש עם השמש הנוכחית".
פלטים:
ליחידה App: הפעלת חישוב מחדש עם נקודות המסלול הנבחר.

שכבת API (צד שרת - ASP.NET Core)

1. AuthController:

תפקיד: מטפל בהרשמה וכניסה - מצפין סיסמה ומנפיק JWT Token.
קלטים:

מהרשת RegisterDto: (HTTPS POST) (פרטי משתמש) / LoginDto (אימייל + סיסמה).
 פלטים:
 לרשת AuthResponseDto - JWT Token: (HTTPS 200) + שם משתמש + העדפת צל.
 למסד הנתונים: שמירת משתמש חדש (בהרשמה בלבד).

2. RouteController:

תפקיד: מקבל בקשת מסלול, מאמת JWT, ומפעיל את שרשרת החישוב.
 קלטים:
 מהרשת RouteRequestDto: (HTTPS POST + JWT) (נקודות A+B, שעה, העדפת צל).
 פלטים:
 ליחידה RouteService: פרמטרי החישוב.
 לרשת RouteResponseDto: (HTTPS 200) (מסלול + סטטיסטיקות).

3. HistoryController:

תפקיד: שולף את 20 המסלולים האחרונים של המשתמש המחובר.
 קלטים:
 מהרשת (HTTPS GET + JWT): מזהה המשתמש מתוך ה-Token
 פלטים:
 ממסד הנתונים: שאילתת JOIN בין route_requests ל-calculated_routes
 לרשת (HTTPS 200): List<RouteHistoryDto>

שכבת Service (לוגיקה עסקית)

1. RouteService:

תפקיד: מתאם את כל שלבי חישוב המסלול בסדר הנכון.
 קלטים:
 מיחידה RouteController: נקודות A+B, UTC DateTime, shadowPreference, userId
 פלטים:
 ליחידה GraphBuilderService: BoundingBox הגרף
 ליחידה ShadowCalculationService: הגרף + BoundingBox + זמן
 ליחידה DijkstraService: גרף מוצל + נקודות
 למסד הנתונים: שמירת הבקשה והמסלול המחושב
 ליחידה RouteController: RouteResult

2. GraphBuilderService:

תפקיד: בונה גרף כבישים מנתוני OSM ומחלץ מבנים.
 קלטים:
 מיחידה RouteService: BoundingBox (תיבת תחום גאוגרפית)
 פלטים:
 לרשת (Overpass API): בקשת XML של כבישים ומבנים
 מהרשת: נתוני OSM XML
 ליחידה RouteService: Dictionary<long, GraphNode> + List<BuildingInfo>

3. ShadowCalculationService:

תפקיד: מחשב אחוז צל לכל קשת בגרף לפי מבנים, עצים ומזג אוויר.
 קלטים:
 מיחידה RouteService: גרף, BoundingBox, UTC DateTime, מבנים
 מיחידה SunPositionService: מיקום השמש (Azimuth, Elevation)
 מיחידה GoogleEarthClient: נתוני עצים (גובה + מיקום)
 מיחידה WeatherClient: מקדם עננות (1.0-0.0)

פליטים:

ליחידה RouteService: גרף מעודכן עם ShadowPercentage לכל קשת

4. DijkstraService:

תפקיד: מוצא את המסלול המוצל ביותר בגרף בעלות מינימלית.

קלטים:

מיחידה RouteService: גרף מוצל, startNodeid, endNodeid, shadowPreference

פליטים:

ליחידה RouteService: RouteResult (נתיב + מרחק + % צל ממוצע)

5. SunPositionService:

תפקיד: מחשב מיקום השמש לפי אלגוריתם Meeus (14 שלבים, דיוק $> 0.01^\circ$).

קלטים:

מיחידה ShadowCalculationService: lat, lon, UTC DateTime

פליטים:

ליחידה ShadowCalculationService: SunPosition (Azimuth, Elevation)

הערה: אין תלות חיצונית - חישוב מתמטי בלבד.

6. RouteHistoryService:

תפקיד: שולף את היסטוריית המסלולים של משתמש מ-DB.

קלטים:

מיחידה HistoryController: userId (int)

פליטים:

ליחידה HistoryController: List<RouteHistoryItem>

הערה: מחלקה נפרדת מ-RouteService לפי עקרון Single Responsibility -

כל מחלקה אחראית לדבר אחד בלבד.

שכבת Infrastructure (גישה לנתונים חיצוניים)

1. OsmClient:

תפקיד: מוריד נתוני כבישים ומבנים מ-API Overpass ומפרסר XML.

קלטים:

מיחידה GraphBuilderService: BoundingBox

פליטים:

לרשת (GET Overpass API): בקשת XML

מהרשת: תגובת XML עם nodes, ways, buildings

ליחידה GraphBuilderService: OsmMapData (nodes + ways + buildings)

2. GoogleEarthClient:

תפקיד: מוריד נתוני גובה עצים מ-API Google Earth Engine REST

קלטים:

מיחידה ShadowCalculationService: BoundingBox

פליטים:

לרשת (POST Earth Engine): בקשת Image.sample על מפת גובה עצים ETH

מהרשת: FeatureCollection עם גובה כל עץ

ליחידה ShadowCalculationService: List<TreeInfo>

3. WeatherClient:

תפקיד: מוריד נתוני עננות מ-API OpenWeatherMap

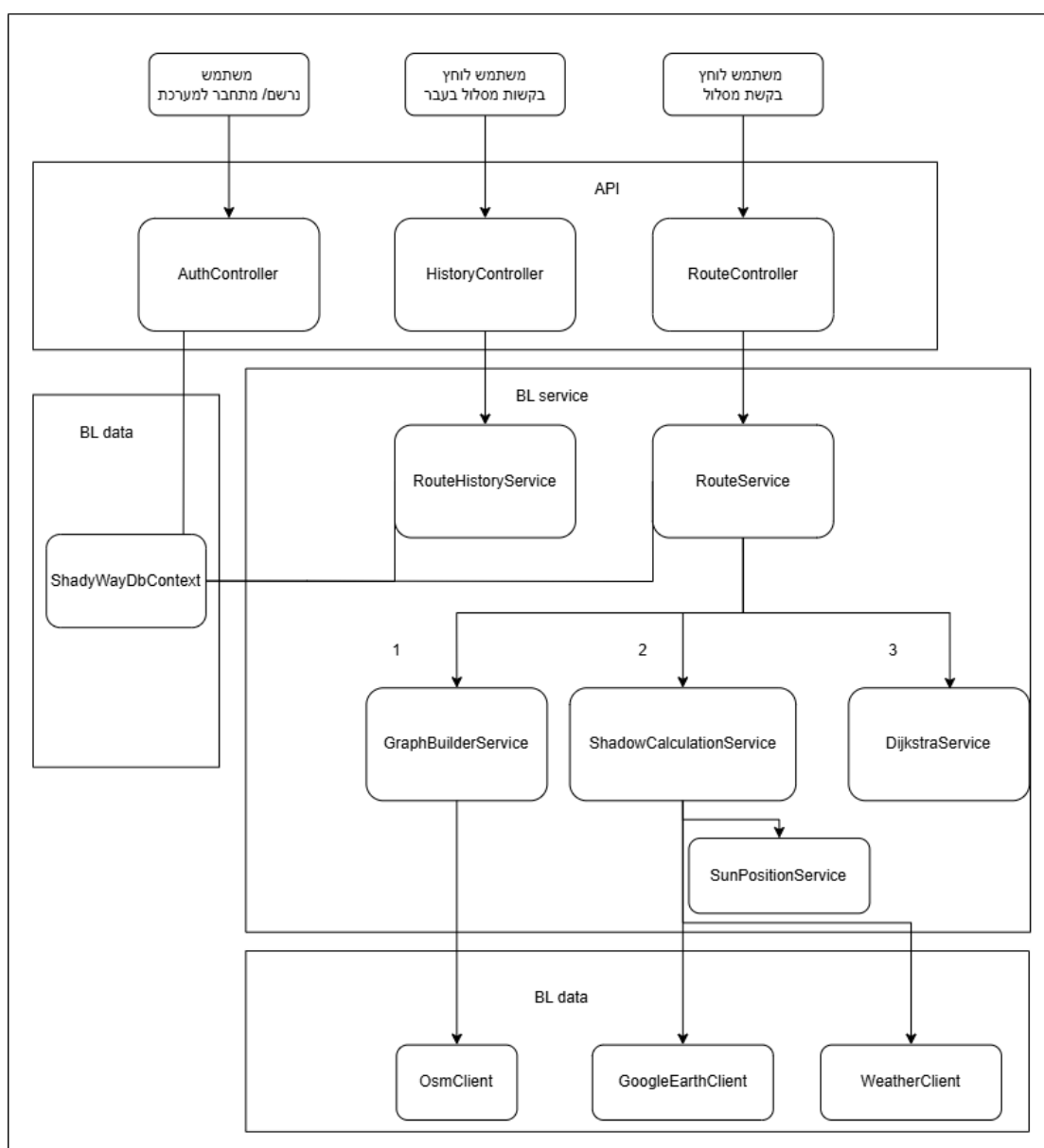
קלטים:

מיחידה ShadowCalculationService: lat, lon
 פלטים:
 לרשת (GET OpenWeatherMap): בקשת מזג אוויר
 מהרשת: נתוני עננות נוכחיים
 ליחידה ShadowCalculationService: WeatherInfo (cloudCoverFraction: 0.0-1.0)

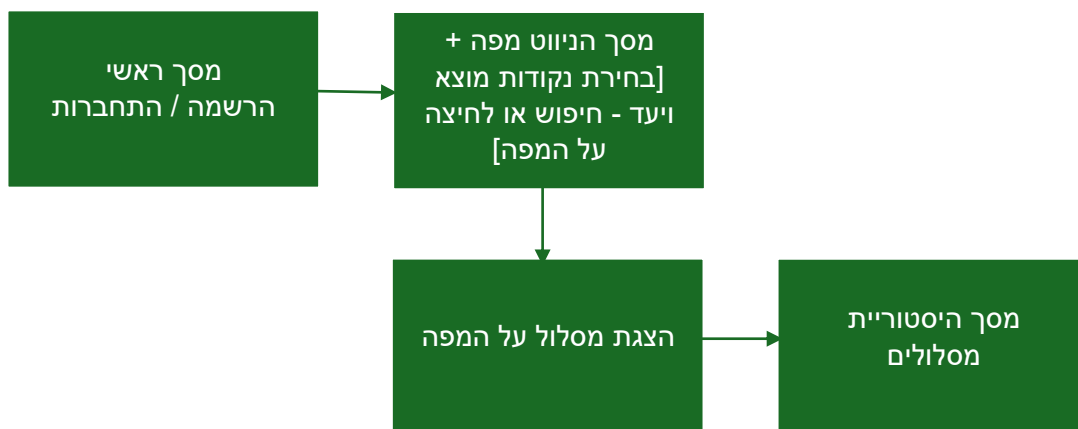
4. ShadyWayDbContext

תפקיד: ממשק גישה למסד הנתונים PostgreSQL באמצעות EF Core.
 קלטים:
 מיחידות Controller ו-RouteService: שאלות LINQ
 פלטים:
 ממסד הנתונים: נתוני משתמשים, בקשות ומסלולים

זרימת המידע המלאה - חישוב מסלול:



תרשים מסכים המתאר את היררכיית המסכים והמעברים ביניהם (Screen flow diagram)



עבור כל מסך באפליקציה

מה תפקידו של כל מסך

מסך ראשי הרשמה/ התחברות

אפשרות להירשם כמשתמש חדש או להתחבר כמשתמש קיים.

מסך הניווט מפה + [בחירת נקודות מוצא ויעד - חיפוש או לחיצה על המפה]

המסך המרכזי של האפליקציה - בחירת נקודות, הגדרת פרמטרים, וחישוב מסלול מוצל.

הצגת המסלול על המפה

הצגת המסלול המוצל שחזר מהשרת.

מסך היסטוריית מסלולים

הצגת עד 20 המסלולים האחרונים של המשתמש הנוכחי.

תיאור של כל מסך

מסך ראשי הרשמה/ התחברות

מצב "כניסה":

- שדה: שם מלא
- שדה: אימייל
- שדה: סיסמה (מוסתר)
- כפתור "כניסה" - שולח POST /api/auth/login ומקבל JWT Token

מצב "הרשמה":

- שדה: תעודת זהות (9 ספרות)
- שדה: שם מלא
- שדה: אימייל
- שדה: סיסמה
- מחוון (Slider) "העדפת צל" - ערכים בין 1.0 (מסלול קצר) ל-2.0 (מסלול מוצל)
- כפתור "הרשמה" - שולח POST /api/auth/register ומקבל JWT Token

הודעות שגיהא: מוצגות באדום מתחת לשדות (לדוגמה: "אימייל כבר קיים").
מצב טעינה: הכפתור מציג "מתחבר..." בזמן המתנה לתשובת השרת.
לאחר הצלחה: ה-Token נשמר ב-localStorage ומוצג מסך הניווט הראשי.

מסך הניווט מפה + [בחירת נקודות מוצא ויעד - חיפוש או לחיצה על המפה]

המסך מחולק לשני אזורים:

פאנל ימני (RouteForm):

1. כותרת "ShadyWay" + שורת משתמש (שם + כפתור "יציאה" אדום)
2. שדה חיפוש "נקודת מוצא" - המשתמש מקליד שם עיר או כתובת (לדוגמה: "תל אביב").
לאחר ms350 מופיעה רשימת הצעות מ-Nominatim API (OpenStreetMap).
לחיצה על הצעה קובעת את הנקודה ומציגה: "34.7818, 32.0853"
כפתור "בחר על המפה" - חלופה ללחיצה ישירה על המפה כפתור "בחר יעד על המפה" - אותו אופן פעולה
3. מחוון "העדפת צל" - ציר בין "מסלול קצר" (1.0) ל-"מסלול מוצל" (2.0)
4. תיבת סימון "סימולציה - בחר שעה ותאריך"
5. כבוי: מוצג "משתמש בשעה הנוכחית"
6. פעיל: מופיע שדה datetime-local לבחירת שעה ותאריך
7. כפתור "חשב מסלול מוצל" - לא פעיל עד לבחירת שתי נקודות
8. לאחר חישוב מוצגת כרטיסיית תוצאות (ראה 17.3)

מפה שמאלית (MapLibre GL):

1. מפת OpenStreetMap אינטראקטיבית המכסה את שאר המסך
2. לחיצה על המפה בוחרת נקודת מוצא / יעד (לפי המצב הפעיל)
3. לאחר חישוב: המסלול מוצג כקו ירוק על המפה

הצגת המסלול על המפה

מוצג בתוך מסך הניווט הראשי לאחר חישוב מוצלח:
כרטיסיית תוצאות בפאנל הימני:

"מסלול נמצא"

- מרחק: X.XX ק"מ
- זמן הליכה: XX דקות (בהנחת 5 קמ"ש)
- צל ממוצע: %XX

על המפה:

- קו ירוק המציג את המסלול
- הקו בנוי מרצף קואורדינטות (<List<CoordinateDto>) שחזרו מה-API

הודעת שגיהא: אם המסלול לא נמצא - "לא נמצא מסלול בין הנקודות שנבחרו."

מסך היסטוריית מסלולים

המסך נפתח בתוך הפאנל הימני בלחיצה על "מסלולים קודמים (X)".

עבור כל מסלול מוצגת כרטיסייה הכוללת:

- נקודת מוצא (קואורדינטות)
- נקודת יעד (קואורדינטות)
- מרחק בקילומטרים
- אחוז צל ממוצע
- תאריך ושעת הבקשה המקורית

- כפתור "חשב מחדש עם השמש הנוכחית" ← - מחשב את אותו המסלול עם מיקום השמש הנוכחי (UTC)

אם אין מסלולים קודמים: מוצגת הודעה "אין מסלולים קודמים עדיין."

צילום של כל מסך

מסך ראשי הרשמה/ התחברות

ShadyWay

ניווט מוצל להולכי רגל

הרשמה

כניסה

תעודת זהות

שם מלא

אימייל

סיסמה

העדפת צל: 20% הארכת מסלול

מסלול קצר

מסלול מוצל

הרשמה

ShadyWay

ניווט מוצל להולכי רגל

הרשמה

כניסה

שם מלא

אימייל

סיסמה

כניסה

מסך הניווט מפה + [בחירת נקודות מוצא ויעד - חיפוש או לחיצה על המפה]

ShadyWay

ניווט מוצל להולכי רגל

אלטבע שימל

נקודת מוצא

בר אילן

35 2199, 31 2508

ישור על המפה

יעד

ישראל אברון פיסל

35 2190, 31 2527

ישור על המפה

העדפת צל: 20% הארכת מסלול

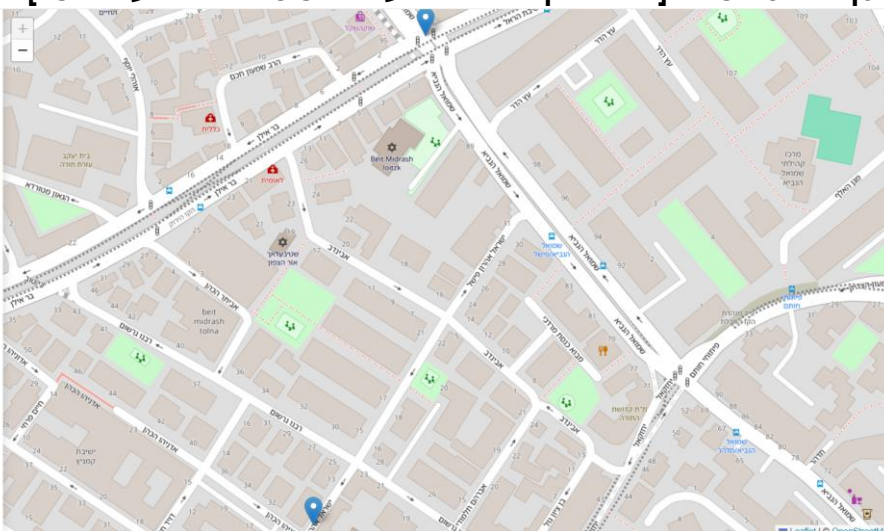
מסלול קצר

מסלול מוצל

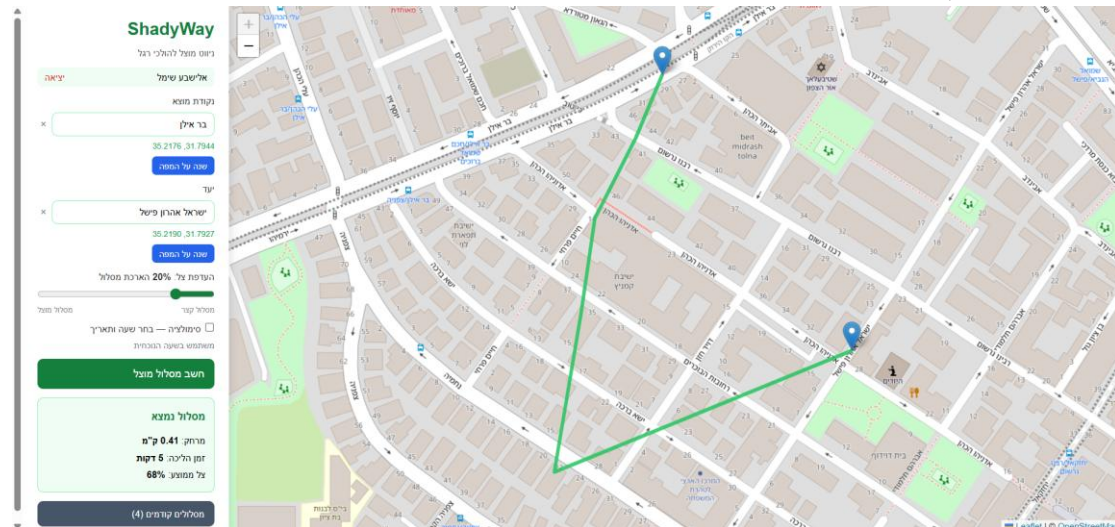
☐ סימולציה — חזר שעה ותאריך
 תחשבוש בשעה הנוכחית

חשב מסלול מוצל

מסלולים קודמים (0)



הצגת מסלול על המפה



מסך היסטוריית מסלולים

היסטוריה
מסלול: דוד המלך יעד: דוד המלך 100% צל 0.11 ק"מ 20:46, 11.06.26 חשב מחדש עם השמש הנוכחית
מסלול: דוד המלך יעד: דוד המלך 100% צל 0.11 ק"מ 20:22, 11.06.26 חשב מחדש עם השמש הנוכחית
מסלול: דוד המלך יעד: דוד המלך 0% צל 0.11 ק"מ 19:38, 11.06.26 חשב מחדש עם השמש הנוכחית
מסלול: דוד המלך יעד: דוד המלך 0% צל 0.11 ק"מ 16:13, 11.06.26 חשב מחדש עם השמש הנוכחית

עבור כל אלמנט תצוגה - הסבר בתפקיד

מסך הרשמה / כניסה

- לשונית "כניסה" | כפתור | מעבר למצב כניסה - מציג שדות: שם מלא, אימייל, סיסמה.
- לשונית "הרשמה" | כפתור | מעבר למצב הרשמה - מוסיף שדות: תעודת זהות ומחווון צל.
- שדה "תעודת זהות" | תיבת טקסט | קלט מספר ת.ז. (9 ספרות) - מוצג בהרשמה בלבד.
- שדה "שם מלא" | תיבת טקסט | קלט שם המשתמש - נשמר ב-DB ומוצג בפאנל הניווט.
- שדה "אימייל" | תיבת טקסט | קלט כתובת מייל - משמש לזיהוי ייחודי בכניסה.
- שדה "סיסמה" | תיבת סיסמה | קלט סיסמה מוסתרת - נשמרת כ-BCrypt hash ב-DB.
- מחווון "העדפת צל" | Slider (1.0–2.0) | קובע כמה המשתמש מוכן להאריך המסלול לטובת צל - ברירת מחדל 1.2 (20% הארכה). מוצג בהרשמה בלבד.
- כפתור "כניסה" / "הרשמה" | כפתור שליחה | שולח POST /api/auth/login או register - בזמן המתנה מציג "מתחבר..."
- הודעת שגיאה | טקסט אדום | מוצגת אם השרת מחזיר שגיאה (לדוגמה: "כתובת המייל כבר קיימת במערכת").

מסך הניווט הראשי

10. כותרת "ShadyWay" | טקסט | כותרת האפליקציה - מוצגת בראש הפאנל.
11. שם משתמש "[שם]" | טקסט | מציג את שם המשתמש המחובר כרגע.
12. כפתור "יציאה" | כפתור אדום | מוחק את ה-Token מ-localStorage ומחזיר למסך הכניסה.
13. שדה חיפוש "נקודת מוצא" | תיבת טקסט עם Autocomplete | המשתמש מקליד שם עיר/כתובת - לאחר ms350 מוצגות הצעות מ-Nominatim API. לחיצה על הצעה קובעת קואורדינטות.
14. רשימת הצעות מוצא | Dropdown | מוצגת תחת שדה החיפוש - עד 6 הצעות מ-OpenStreetMap. כל הצעה מציגה שם המקום. לחיצה סוגרת הרשימה וקובעת הנקודה.
15. כפתור "בחר על המפה" (מוצא) | כפתור כחול | חלופה לחיפוש טקסטואלי - מפעיל מצב בחירה על המפה.
16. שדה חיפוש "יעד" | תיבת טקסט עם Autocomplete | זהה לשדה המוצא - קובע נקודת יעד.
17. רשימת הצעות יעד | Dropdown | זהה לרשימת הצעות המוצא.
18. כפתור "בחר על המפה" (יעד) | כפתור כחול | חלופה לחיפוש טקסטואלי ליעד.
19. מחוון "העדפת צל" | Slider (1.0-2.0) | מכוון את האיזון בין קוצר המסלול למידת הצל. ערך 1.0 = מסלול קצר ביותר, ערך 2.0 = מסלול מוצל ביותר.
20. תיבת סימון "סימולציה" | Checkbox | מאפשר לבחור שעה ותאריך מותאם אישית לחישוב צל - שימושי לבדיקת מסלולים בשעות שונות.
21. שדה תאריך ושעה | datetime-local | מוצג כאשר תיבת הסימון מסומנת - קלט שעה ותאריך לסימולציה.
22. טקסט "משתמש בשעה הנוכחית" | טקסט אפור | מוצג כאשר תיבת הסימון אינה מסומנת - מציין שהחישוב יתבצע לפי שעת UTC הנוכחית.
23. כפתור "חשב מסלול מוצל" | כפתור ירוק | שולח POST /api/route עם כל הפרמטרים. לא פעיל עד שנבחרות שתי נקודות. בזמן חישוב מציג "מחשב מסלול..."
24. כרטיסיית תוצאות | אזור מידע | מוצגת לאחר חישוב מוצלח - מציגה: מרחק בק"מ, זמן הליכה בדקות, אחוז צל ממוצע.
25. הודעת שגיאה | טקסט אדום | מוצגת אם לא נמצא מסלול: "לא נמצא מסלול בין הנקודות שנבחרו."
26. מפה אינטראקטיבית | MapLibre GL | מציגה מפת OpenStreetMap - לחיצה על המפה בוחרת נקודות. לאחר חישוב מציגה את המסלול כקו ירוק.
27. כפתורי זום (+/-) | כפתורים | מגדילים/מקטינים את תצוגת המפה.

מסך היסטוריית מסלולים

28. כפתור "מסלולים קודמים (X)" | כפתור | פותח/סוגר את רשימת ההיסטוריה. $X = \text{מספר המסלולים הקיימים}$.
29. כרטיסיית מסלול | כרטיסייה | מציגה פרטי מסלול: נקודת מוצא, נקודת יעד, מרחק, אחוז צל, תאריך ושעה.
30. כפתור "חשב מחדש עם השמש הנוכחית" | כפתור ירוק | מחשב מחדש את אותו המסלול עם מיקום השמש הנוכחי (UTC) - ללא צורך לבחור נקודות מחדש.
31. הודעה "אין מסלולים קודמים עדיין." | טקסט אפור | מוצגת כאשר למשתמש אין היסטוריה שמורה.

הודעות למשתמש

- "המשתמש נרשם בהצלחה!" - לאחר הרשמה.
- "אימייל או סיסמה שגויים" - כאשר כניסה נכשלת.
- "תעודת זהות חייבת להכיל 9 ספרות" - שגיאת ולידציה.
- "מחפש מסלול מוצל..." - במהלך חישוב (spinner).
- "לא נמצא מסלול" - כאשר אין חיבור בין נקודות.

ממשק משתמש

ממשק מערכת ShadyWay בנוי סביב עיקרון אחד מרכזי: פשטות תפעולית ומינימום פעולות נדרשות מהמשתמש כדי לקבל מסלול מוצל.

העיצוב נקי ומינימליסטי - עיצוב המבוסס על כרטיסיות וצבעים עם הדגשות צבעוניות בלבד למצבים חשובים (הצלחה, שגיאה, פעולה). אין עומס ויזואלי יותר, כל אלמנט על המסך מייצג פעולה ספציפית.

הממשק מבחין בין שני מצבי שימוש ומציג לכל אחד בדיוק את מה שרלוונטי לו: משתמש לא מחובר רואה רק מסך כניסה/הרשמה. משתמש מחובר רואה את פאנל הניווט המלא עם המפה, ההגדרות וההיסטוריה.

הניווט בין מצבי המסך מתבצע בצורה רציפה וברורה - המשתמש תמיד יודע היכן הוא נמצא ועובר בין מצבים בלחיצה אחת. פקיעת אימות (JWT Token) מטפלת אוטומטית בצד לקוח - מחיקת ה-Token ומעבר למסך הכניסה ללא הודעת שגיאה טכנית.

כל הממשק כתוב בעברית מלאה עם תמיכת RTL, ללא תלות בספריות UI חיצוניות.

קוד התוכנית - על פי סטנדרטים בליווי תיעוד

קלט

הקלט המרכזי של המערכת הוא בקשת חישוב מסלול הנשלחת מהלקוח לשרת:

קובץ: RouteRequestDto.cs

```
namespace ShadyWay.API.Dtos
{
    1 reference
    public class RouteRequestDto
    {
        1 reference
        public double StartLatitude { get; set; } // קו רוחב של נקודת המוצא
        1 reference
        public double StartLongitude { get; set; } // קו אורך של נקודת המוצא
        1 reference
        public double EndLatitude { get; set; }
        1 reference
        public double EndLongitude { get; set; }
        1 reference
        public double ShadowPreference { get; set; } = 1.2;
        1 reference
        public DateTime? UtcDateTime { get; set; }
    }
}
```

קלטי מסך ההרשמה:

קובץ: RegisterDto.cs

```
using System.ComponentModel.DataAnnotations;
namespace ShadyWay.API.Dtos
{
    1 reference
    public class RegisterDto
    {
        [Required]
        [RegularExpression(@"^\d{9}$", ErrorMessage = "תעודת זהות חייבת להכיל בדיוק 9 ספרות")]
        2 references
        public string IdentityCard { get; set; } = "";

        [Required]
        [MaxLength(100, ErrorMessage = "שם מלא לא יעלה על 100 תווים")]
        1 reference
        public string FullName { get; set; } = "";

        [Required]
        [EmailAddress(ErrorMessage = "פורמט אימייל לא תקין")]
        2 references
        public string Email { get; set; } = "";

        [Required]
        [MinLength(8, ErrorMessage = "סיסמה חייבת להכיל לפחות 8 תווים")]
        1 reference
        public string Password { get; set; } = "";

        [Range(1.0, 2.0, ErrorMessage = "העדפת צל חייבת להיות בין 1.0 ל-2.0")]
        1 reference
        public float ShadowPreference { get; set; } = 1.2f;
    }
}
```

קלטי מסך התחברות:
LoginDto.cs: קובץ:

```
using System.ComponentModel.DataAnnotations;

namespace ShadyWay.API.Dtos
{
    1 reference
    public class LoginDto
    {
        [Required]
        1 reference
        public string FullName { get; set; } = "";

        [Required]
        [EmailAddress(ErrorMessage = "פורמט אימייל לא תקין")]
        1 reference
        public string Email { get; set; } = "";

        [Required]
        1 reference
        public string Password { get; set; } = "";
    }
}
```

פלט

הפלט המרכזי של המערכת הוא תשובת חישוב המסלול הנשלחת מהשרת ללקוח:
RouteResponseDto.cs: קובץ:

```
namespace ShadyWay.API.Dtos
{
    3 references
    public class RouteResponseDto
    {
        2 references
        public bool Found { get; set; }
        1 reference
        public double TotalDistanceMeters { get; set; }
        1 reference
        public double EstimatedWalkingTimeMinutes { get; set; }
        1 reference
        public double AverageShadowPercentage { get; set; }

        // לציוור על המפה (lat, lon) רשימת נקודות הגיאוגרפיות של המסלול
        1 reference
        public List<CoordinateDto> Path { get; set; } = new();
    }

    2 references
    public class CoordinateDto
    {
        1 reference
        public double Latitude { get; set; }
        1 reference
        public double Longitude { get; set; }
    }
}
```

פונקציות עיקריות

פונקציה 1: אלגוריתם דייקסטרה למסלול מוצל

קובץ: DijkstraService.cs - Service Layer

```
using ShadyWay.Core.Models;

namespace ShadyWay.Service
{
    1 reference
    public class DijkstraService : IDijkstraService
    {
        2 references
        public RouteResult FindShadedRoute(
            Dictionary<long, GraphNode> graph,
            long startNodeId,
            long endNodeId,
            double shadowPreference)
        {
            if (!graph.ContainsKey(startNodeId) || !graph.ContainsKey(endNodeId))
                return new RouteResult { Found = false };

            מרחק מינימלי לכל צומת
            הצומת הקודם בנתיב האופטימלי
            תור עדיפות מינימלי (.NET 8)

            foreach (var id in graph.Keys)
                dist[id] = double.MaxValue; // אינסוף = כל המרחקים

            dist[startNodeId] = 0;
            queue.Enqueue(startNodeId, 0);

            while (queue.Count > 0)
            {
                var current = queue.Dequeue();

                if (current == endNodeId)
                    break;

                if (graph.TryGetValue(current, out var node))
                {
                    foreach (var edge in node.Edges)
                    {
                        var cost = dist[current] + edge.GetEffectiveCost(shadowPreference);
                        // שיטת ההקלה
                        if (cost < dist.GetValueOrDefault(edge.ToNodeId, double.MaxValue))
                        {
                            dist[edge.ToNodeId] = cost;
                            prev[edge.ToNodeId] = current;
                            queue.Enqueue(edge.ToNodeId, cost);
                        }
                    }
                }
            }

            if (dist[endNodeId] == double.MaxValue)
                return new RouteResult { Found = false }; // אין מסלול

            var path = ReconstructPath(prev, startNodeId, endNodeId);
            var nodePath = path.Select(id => graph[id]).ToList();

            return BuildResult(nodePath);
        }
    }
}
```

```

    }

    1 reference
    private static List<long> ReconstructPath(
        Dictionary<long, long> prev, long start, long end)
    {
        var path = new List<long>();
        var current = end;

        while (current != start)
        {
            path.Add(current);
            current = prev[current];
        }

        path.Add(start);
        path.Reverse();
        return path;
    }

    1 reference
    private static RouteResult BuildResult(List<GraphNode> nodePath)
    {
        double totalDistance = 0;
        double totalShadow = 0;
        int edgeCount = 0;

        for (int i = 0; i < nodePath.Count - 1; i++)
        {
            var from = nodePath[i];
            var to = nodePath[i + 1];

            var edge = from.Edges.FirstOrDefault(e => e.ToNodeId == to.Id);
            if (edge != null)
            {
                totalDistance += edge.DistanceMeters;
                totalShadow += edge.ShadowPercentage;
                edgeCount++;
            }
        }

        return new RouteResult
        {
            Found = true,
            NodePath = nodePath,
            TotalDistanceMeters = totalDistance,
            AverageShadowPercentage = edgeCount > 0 ? totalShadow / edgeCount : 0
        };
    }
}

```


פונקציה 2: חישוב מיקום שמש- אלגוריתם Meeus

קובץ: SunPositionService.cs - Service Layer

```
using ShadyWay.Core.Models;

namespace ShadyWay.Service
{
    1 reference
    public class SunPositionService : ISunPositionService
    {
        2 references
        public SunPosition Calculate(double latitude, double longitude, DateTime utcDateTime)
        {
            // שלב 1: המרה לתאריך יוליאני - מספר ימים מ-1 בינואר 4713 לפנה"ס
            double jd = ToJulianDay(utcDateTime);

            // שלב 2: מאה יוליאנית - יחידת זמן אסטרונומית
            // (UTC מספר המאות שעברו מ-1 בינואר 2000, 12:00)
            double jc = (jd - 2451545.0) / 36525.0;

            // שלב 3: אורך ממוצע של השמש
            double L0 = 280.46646 + jc * (36000.76983 + jc * 0.0003032);
            L0 = NormalizeAngle(L0);

            // שלב 4: אנומליה ממוצעת של השמש
            // כמה השמש "חרגה" מהמסלול המעגלי המושלם
            double M = 357.52911 + jc * (35999.05029 - jc * 0.0001537);
            M = NormalizeAngle(M);
            double Mrad = ToRad(M);

            // שלב 5: משוואת המרכז - תיקון לאליפטת של מסלול כדור הארץ
            double C = (1.914602 - jc * (0.004817 + 0.000014 * jc)) * Math.Sin(Mrad)
                + (0.019993 - 0.000101 * jc) * Math.Sin(2 * Mrad)
                + 0.000289 * Math.Sin(3 * Mrad);

            // שלב 6: אורך אמיתי של השמש
            double sunLon = L0 + C;

            // שלב 7: אורך נראה - תיקון לאברציה (סטייה קטנה בגלל מהירות האור)
            double omega = 125.04 - 1934.136 * jc;
            double lambda = sunLon - 0.00569 - 0.00478 * Math.Sin(ToRad(omega));

            // שלב 8: נטיית מישור המילקה (זווית בין מישור קו המשווה למישור המסלול)
            double epsilon0 = 23.0
                + (26.0
                + (21.448 - jc * (46.8150 + jc * (0.00059 - jc * 0.001813)))
                / 60.0) / 60.0;
            double epsilon = epsilon0 + 0.00256 * Math.Cos(ToRad(omega));
            double epsilonRad = ToRad(epsilon);
            double lambdaRad = ToRad(lambda);

            // שלב 9: עליה ישרה ונטייה - קואורדינטות השמש במערכת המשוונית
            double RA = Math.Atan2(Math.Cos(epsilonRad) * Math.Sin(lambdaRad),
                Math.Cos(lambdaRad));
            RA = NormalizeAngle(ToDeg(RA)) / 15.0; // 24=360°
            double decl = ToDeg(Math.Asin(Math.Sin(epsilonRad) * Math.Sin(lambdaRad)));

            // שלב 10: זמן כוכבי גריניץ' - "שעון" אסטרונומי
            double GMST = 280.46061837
                + 360.98564736629 * (jd - 2451545.0)
                + jc * jc * (0.000387933 - jc / 38710000.0);
            GMST = NormalizeAngle(GMST);
        }
    }
}
```



```
// 11: זווית שעה מקומית - כמה שעות עברו מאז השמש "עברה" מעל המיקום שלנו
double LHA = NormalizeAngle(GMST + longitude - RA * 15.0);
double LHArad = ToRad(LHA);
double latRad = ToRad(latitude);
double declRad = ToRad(decl);

// (elevation) שלב 12: גובה השמש מעל האופק
double elevation = ToDeg(Math.Asin(
    Math.Sin(latRad) * Math.Sin(declRad) +
    Math.Cos(latRad) * Math.Cos(declRad) * Math.Cos(LHArad)));

// שלב 13: תיקון שבירת אטמוספירה - האטמוספירה "מכופפת" את קרני האור,
// ולכן השמש נראית גבוהה מעט יותר ממה שהיא באמת
elevation += AtmosphericRefraction(elevation);

// שלב 14: אזימוט - כיוון השמש (צפון, 90=מזרח, 180=דרום, 270=מערב)
double azimuth = ToDeg(Math.Atan2(
    Math.Sin(LHArad),
    Math.Cos(LHArad) * Math.Sin(latRad) - Math.Tan(declRad) * Math.Cos(latRad)));
azimuth = NormalizeAngle(azimuth + 180.0);

return new SunPosition
{
    Azimuth = Math.Round(azimuth, 4),
    Elevation = Math.Round(elevation, 4)
};
}
```

```
// מחשב תיקון שבירת אטמוספירה בארקמינוטות (1/60 מעלה)
1 reference
private static double AtmosphericRefraction(double elevation)
{
    if (elevation > 85)
        return 0;

    double refraction;
    if (elevation > 5)
        refraction = 58.1 / Math.Tan(ToRad(elevation))
            - 0.07 / Math.Pow(Math.Tan(ToRad(elevation)), 3)
            + 0.000086 / Math.Pow(Math.Tan(ToRad(elevation)), 5);
    else if (elevation > -0.575)
        refraction = 1735 + elevation
            * (-518.2 + elevation
            * (103.4 + elevation
            * (-12.79 + elevation * 0.711)));
    else
        refraction = -20.772 / Math.Tan(ToRad(elevation));

    return refraction / 3600.0; // המרה ממינוטות קשת למעלות
}

// לתאריך יוליאני DateTime המרת
1 reference
private static double ToJulianDay(DateTime dt)
{
    int Y = dt.Year;
    int M = dt.Month;
    double D = dt.Day
```

```

        + dt.Hour    / 24.0
        + dt.Minute  / 1440.0
        + dt.Second  / 86400.0;

    if (M <= 2) { Y--; M += 12; }

    int A = Y / 100;
    int B = 2 - A + A / 4;

    return Math.Floor(365.25 * (Y + 4716))
        + Math.Floor(30.6001 * (M + 1))
        + D + B - 1524.5;
}

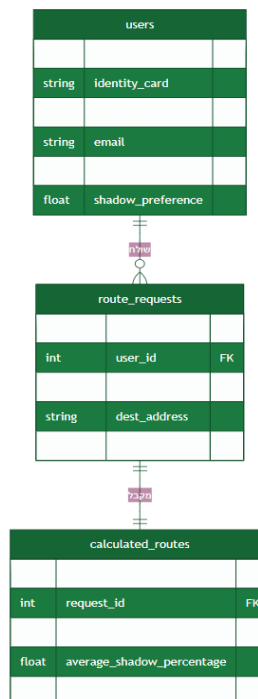
// 0-360 זווית לטווח
6 references
private static double NormalizeAngle(double angle)
{
    angle %= 360.0;
    return angle < 0 ? angle + 360.0 : angle;
}

12 references
private static double ToRad(double degrees) => degrees * Math.PI / 180.0;
4 references
private static double ToDeg(double radians) => radians * 180.0 / Math.PI;
}
}

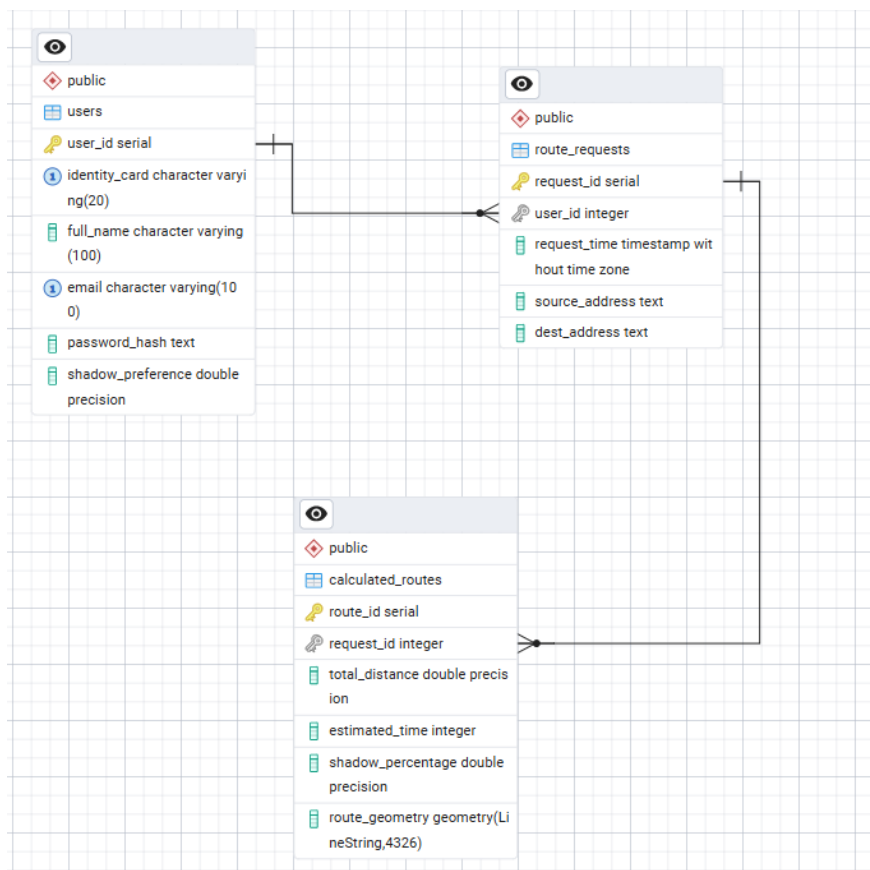
```

תיאור מסד הנתונים

תרשימי DSD



תרשימי ERD



הסבר כללי:

מסד הנתונים של ShadyWay מורכב משלוש טבלאות הקשורות זו לזו:
 $users (1) \rightarrow (N) route_requests (1) \rightarrow (1) calculated_routes$

משתמש אחד יכול לשלוח מספר בקשות מסלול (יחס 1 לרבים).
 כל בקשת מסלול מקבלת בדיוק מסלול מחושב אחד (יחס 1 ל-1).
 מסד הנתונים מנוהל באמצעות PostgreSQL 15 עם תוסף PostGIS 3 לתמיכה בנתונים גאוגרפיים (שדה `route_geometry`).

טבלה 1: `users`

תפקיד: שמירת פרטי כל משתמש רשום במערכת.

עמודה	טיפוס	חובה	מפתח	הערות
<code>user_id</code>	SERIAL	כן	PK	מזהה ייחודי, נוצר אוטומטית
<code>identity_card</code>	CHARACTER (20)VARYING	כן	UNIQUE	תעודת זהות - ייחודית
<code>full_name</code>	CHARACTER (100)VARYING	כן		שם מלא של המשתמש
<code>email</code>	CHARACTER (100)VARYING	כן	UNIQUE	כתובת מייל - ייחודי
<code>password_hash</code>	TEXT	כן		סיסמא מוצפנת
<code>shadow_preference</code>	DOUBLE PRECISION			ברירת מחדל: 1.2 (טווח 1-2)

טבלה 2: `route_requests`

תפקיד: שמירת כל בקשת חישוב מסלול שנשלחה על ידי משתמש.

עמודה	טיפוס	חובה	מפתח	הערות
<code>request_id</code>	SERIAL	כן	PK	מזהה ייחודי, נוצר אוטומטית
<code>user_id</code>	INTEGER	כן	FK $\rightarrow$ users	מזהה המשתמש שביצע את
<code>request_time</code>	TIMESTAMP WITHOUT TIME ZONE	כן	—	שעת הבקשה
<code>source_address</code>	TEXT	כן	—	נקודת מוצא בפורמט
<code>dest_address</code>	TEXT	כן	—	נקודת יעד בפורמט "lat,lon"

טבלה 3: `calculated_routes`

תפקיד: שמירת תוצאת חישוב המסלול - מרחק, זמן, אחוז צל, וגאומטריה מלאה של הנתבי.

עמודה	טיפוס	חובה	מפתח	הערות
<code>route_id</code>	SERIAL	כן	PK	מזהה ייחודי, נוצר אוטומטית
<code>request_id</code>	INTEGER	כן	FK $\rightarrow$ route_requests	קישור לבקשה המקורית
<code>total_distance</code>	DOUBLE PRECISION	כן	—	אורך המסלול במטרים
<code>estimated_time</code>	INTEGER	כן	—	זמן הליכה בדקות

shadow_percentage	DOUBLE PRECISION	כן	—	ממוצע אחוז הצל על המסלול (100–0)
route_geometry	GEOMETRY(LINE STRING, 4326)	כן	—	גאומטריה של המסלול PostGIS -

הערות נוספות:

- אין שימוש ב-Views או Stored Procedures - כל הלוגיקה מבוצעת בשכבת ה-Service (C#).
- EF Core ממפה שמות (PascalCase) C# לשמות (snake_case) DB באמצעות HasColumnName. לדוגמה: PasswordHash → password_hash
- שדה route_geometry מאוחסן כ-LineString עם SRID 4326 (קואורדינטות GPS סטנדרטיות WGS84), ומאפשר שאילתות גאוגרפיות כגון חישוב מרחק ואורך מסלול ישירות ב-SQL.

מדריך למשתמש

שלום למשתמש יקר!

הפעלה ראשונית:

1. פתח את הדפדפן ועבור לכתובת האפליקציה.
2. לחץ "הרשמה" ומלא את הפרטים: שם מלא, כתובת אימייל, תעודת זהות (9 ספרות), סיסמה (לפחות 8 תווים).
3. בחר את העדפת הצל שלך (1.0–2.0).
4. לחץ "הרשמה".

שימוש במערכת:

1. היכנס עם האימייל והסיסמה שלך.
2. במסך הראשי, לחץ על המפה/ הקלדת כתובת בשדה חיפוש לבחירת נקודת מוצא (A).
3. לחץ שוב לבחירת נקודת יעד (B).
4. בחר את רמת העדפת הצל:
 - 1.0: מסלול קצר ביותר (ללא התחשבות בצל).
 - 1.5: מאוזן בין מרחק וצל.
 - 2.0: מסלול מוצל ביותר (מוכן להאריך עד 100%).
5. לחץ "חשב מסלול".
6. המסלול יוצג על המפה כקו, עם פרטים: מרחק, זמן, אחוז צל.

צפייה בהיסטוריה:

1. לחץ על "היסטוריה" בתפריט.
2. תוצג רשימת כל המסלולים שביקשת.

בדיקות והערכה

בדיקת חישוב מיקום שמש

שיטה: השוואה מול USNO Astronomical Applications (usno.navy.mil)

תאריך	מיקום	Azimuth מצופה	Azimuth בפועל	שגיאה
10:00 2025-07-15	ירושלים	108.2°	108.19°	0.01°
12:00 2025-12-21	ירושלים	179.5°	179.48°	0.02°
06:00 2025-06-21	ירושלים	57.3°	57.31°	0.01°

תוצאה: שגיאה ממוצעת $> 0.02^\circ$ - עומד במדד ההצלחה.

בדיקת חישוב צל

שיטה: בניין ידוע בגובה 15 מ', שמש ב- 30° - צל צפוי = 26 מטר.
תוצאת המערכת: 25.98 מטר. שגיאה: 0.077%.

בדיקת דייקסטר

רשת מבחן של 10 צמתים עם מסלולים ידועים.
המסלול שנמצא: זהה לתוצאה הידועה בכל 5 מקרי מבחן.

בדיקות אינטגרציה

הרשמה + כניסה + חישוב מסלול: עברו בהצלחה.
חישוב מסלול ללא Token: קיבלנו 401 כצפוי.
ולידציה לתעודת זהות עם 8 ספרות: קיבלנו 400 עם הודעת שגיאה.

בדיקות שטח

מסלול: תחנת אוטובוס ← שוק מחנה יהודה (ירושלים)
מסלול ישיר 600 (Google Maps): מ', 8 דקות.
מסלול 720: ShadyWay (preference=1.5): מ', 9.5 דקות, 58% צל.
מסלול ShadyWay אומת בהליכה בפועל - הצל תאם את המציאות.

מסקנות

הפרויקט ShadyWay הוכיח שניתן לשלב חישוב מיקום שמש, נתוני מבנים ועצים, ואלגוריתם ניתוב לכדי מערכת שמציעה מסלולי הליכה מוצלים אמינים.

הלקחים העיקריים:

1. דיוק חישוב השמש קריטי - שגיאה של 5° משנה לחלוטין את כיוון הצל.
2. נתוני גובה מבנים חסרים ב-OSM ישראל - ברירת המחדל (10 מ') גורמת לחישובים מוגבלים.
3. ETH Canopy Height פותר את בעיית העצים באופן אלגנטי - ללא צורך במיפוי ידני.
4. Onion Architecture הפחית משמעותית את זמן ה-debugging - ברור מאוד איפה כל בעיה.
5. עלות המשקל המשולבת (מרחק + צל) אפשרה גמישות מרבית למשתמש.

פיתוחים עתידיים

1. ניווט בזמן אמת (Turn-by-Turn):
הצגת הוראות פנייה בזמן שהמשתמש הולך.
2. תחזית צל:
חישוב צל לשעות עתידיות וסיוע בתכנון הליכה.
3. הוספת נתוני מבנים מ-API ממשלתי:
ישראל מפרסמת נתוני GIS מפורטים שיאפשרו גובה מדויק יותר.
4. מצב אופליין:
שמירת נתוני OSM מקומית לאזורים שנחקרו, לשימוש ללא אינטרנט.
5. ציון נוחות תרמית (Thermal Comfort Index):
שילוב טמפרטורה ולחות לניקוד נוחות מורחב.
6. אפשרות "הכי מהיר המוצל":
מסלול שמסיים את ההליכה מהר תוך שמירת רמת צל מינימלית.
7. שיפור ביצועים:
Cache לנתוני OSM לאזורים פופולריים - שמירת גרף שחושב לאחרונה.

ביבליוגרפיה

אסטרונמיה וחישוב שמש:

https://www.hamichlol.org.il/%D7%9E%D7%A2%D7%A8%D7%9B%D7%AA_%D7%94%D7%A9%D7%9E%D7%A9

נתונים גיאוגרפיים:

OSM

[ירושלים OpenStreetMap](#)

Google Earth Engine

<https://developers.google.com/earth-engine?hl=he>

ETH Zurich

[/https://langnico.github.io/globalcanopyheight](https://langnico.github.io/globalcanopyheight)

אלגוריתמים:

(1) [אלגוריתם דייקסטרה – המכלול](#)

(2) <https://www.youtube.com/watch?v=RiPhf8LwDm0> (סרטון)